AI VIET NAM
@aivietnam.edu.vn

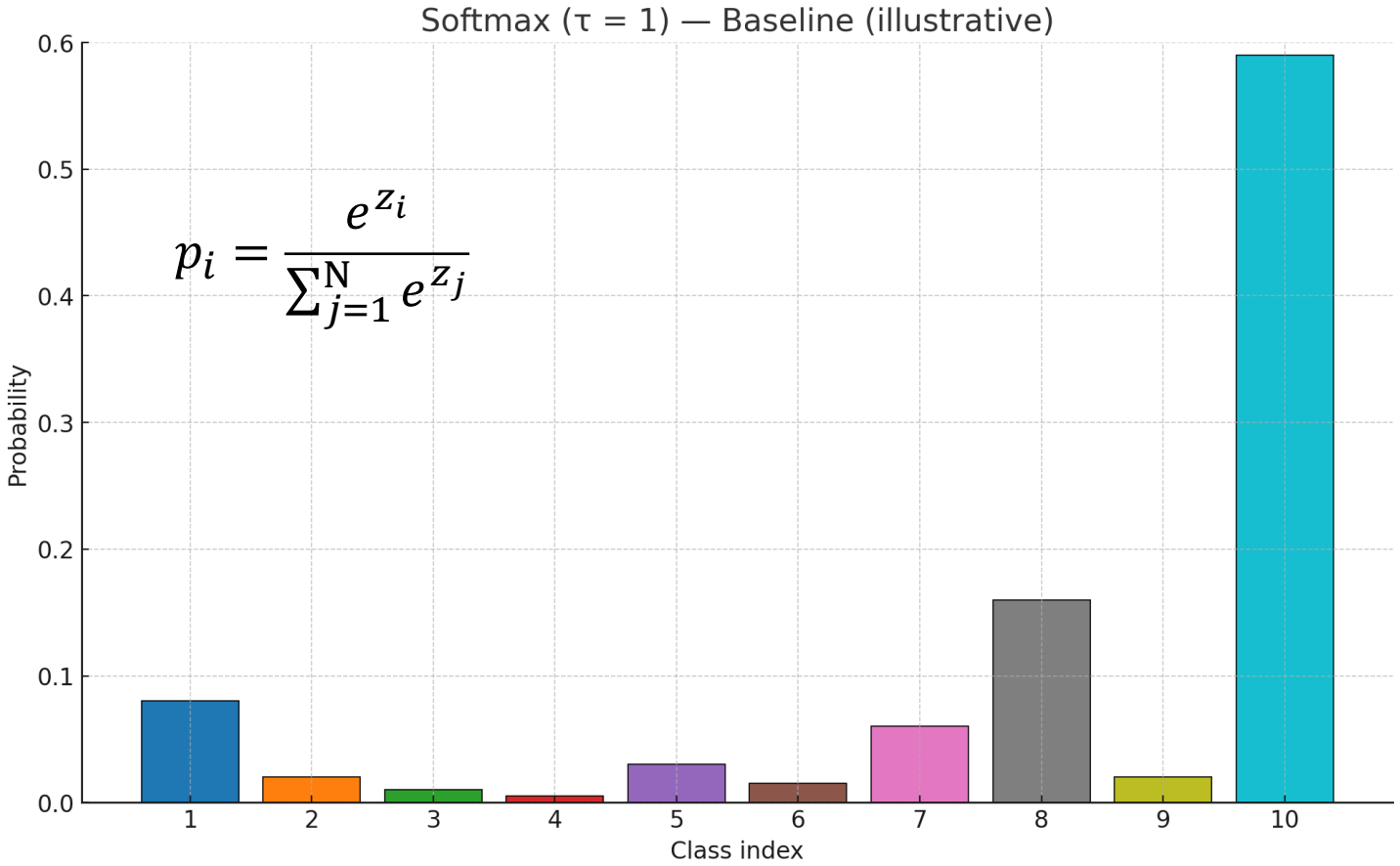
Softmax Regression

Exercise

MSc. Dương Đình Thắng

TA Nguyễn Đặng Duy Tân

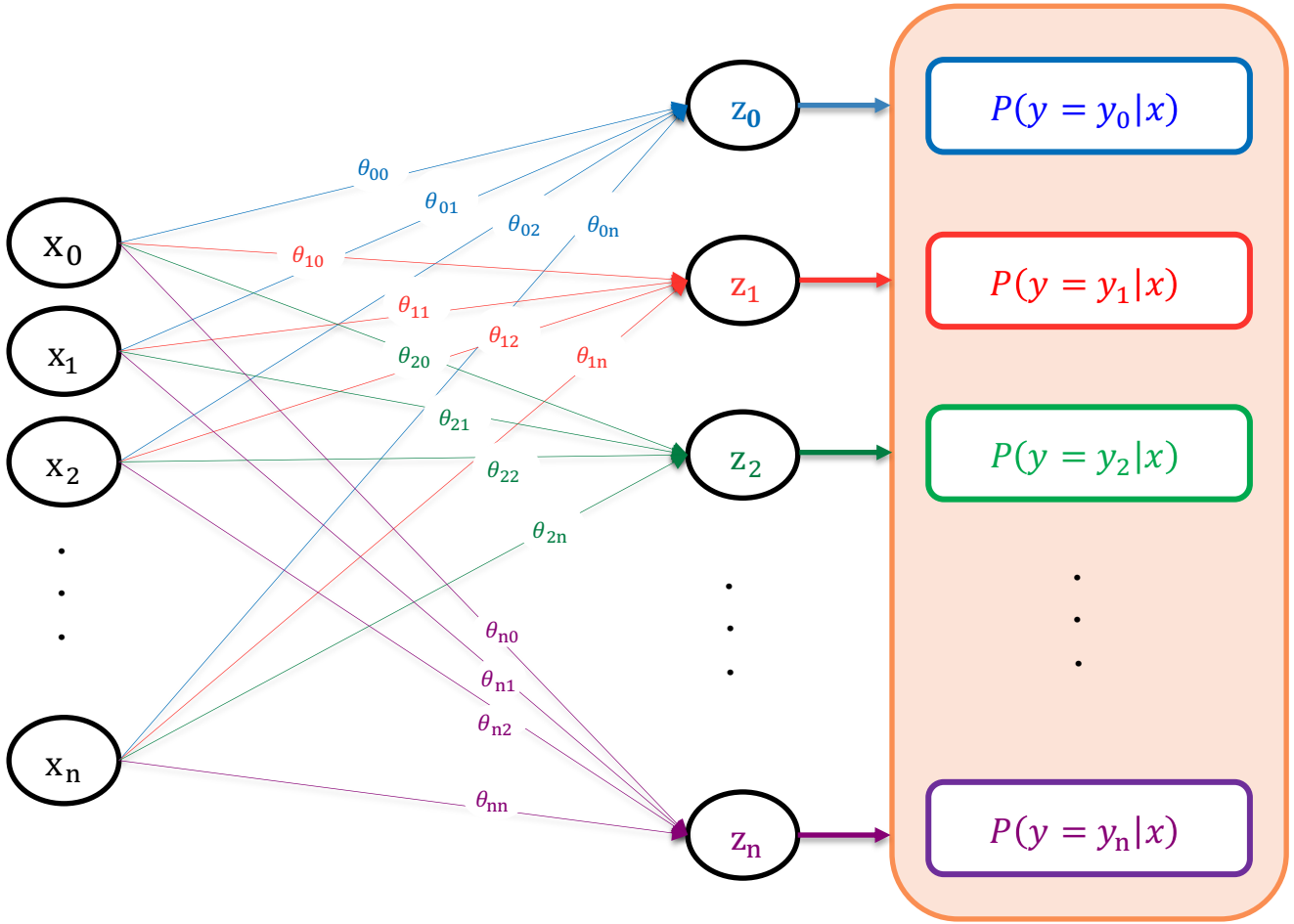
Motivation



Why does this topic matter?

- Real-world problems usually have more than two possible outcomes, not just yes or no.
- A simple idea that shows up again and again in more advanced models later on (e.g. attention mechanism, next-token prediction…).

Objectives



In this session, we will discuss:

1. How Softmax Regression extends Logistic Regression to multiclass classification.
2. How the Softmax Function and SGD are used to train Softmax Regression.
3. How Softmax Regression is implemented for Card Fraud Detection and Sentiment Analysis using NumPy and PyTorch.

Outline

- Recap
- Exercise 01
- Exercise 02
- Question

Recap

Recap

❖ Getting Started

Input



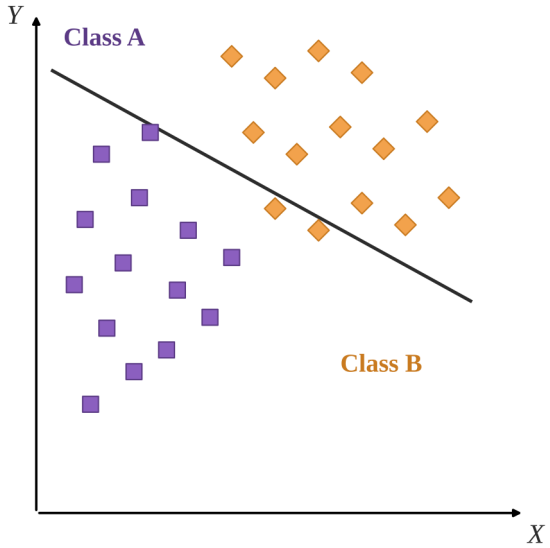
An email

Classification problem

Classification Program

Output

- Spam
- Not Spam

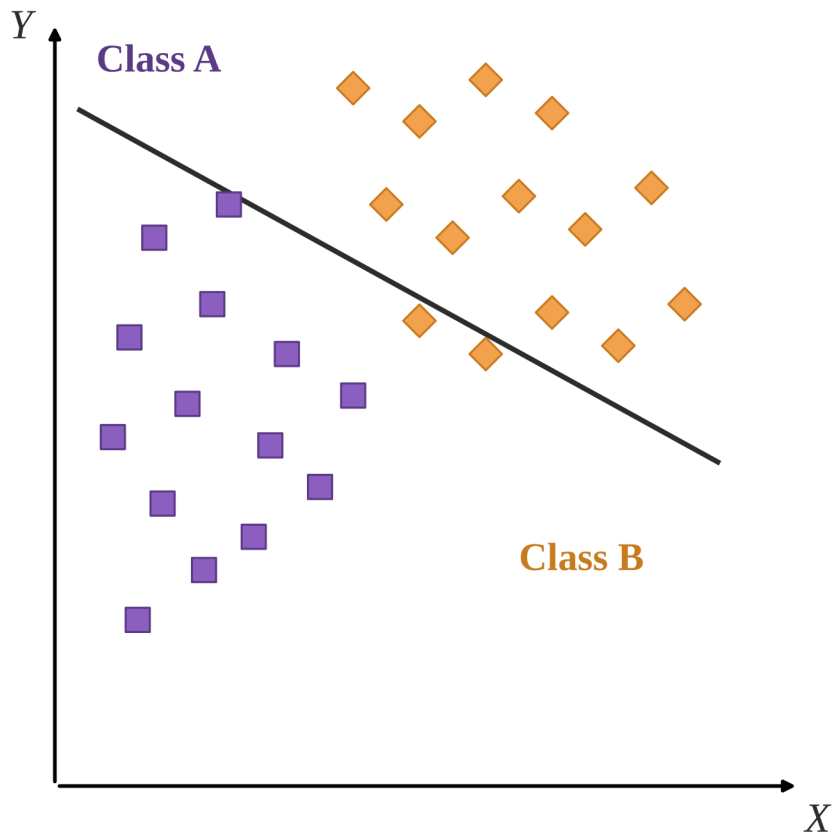


Output is categorical. E.g:

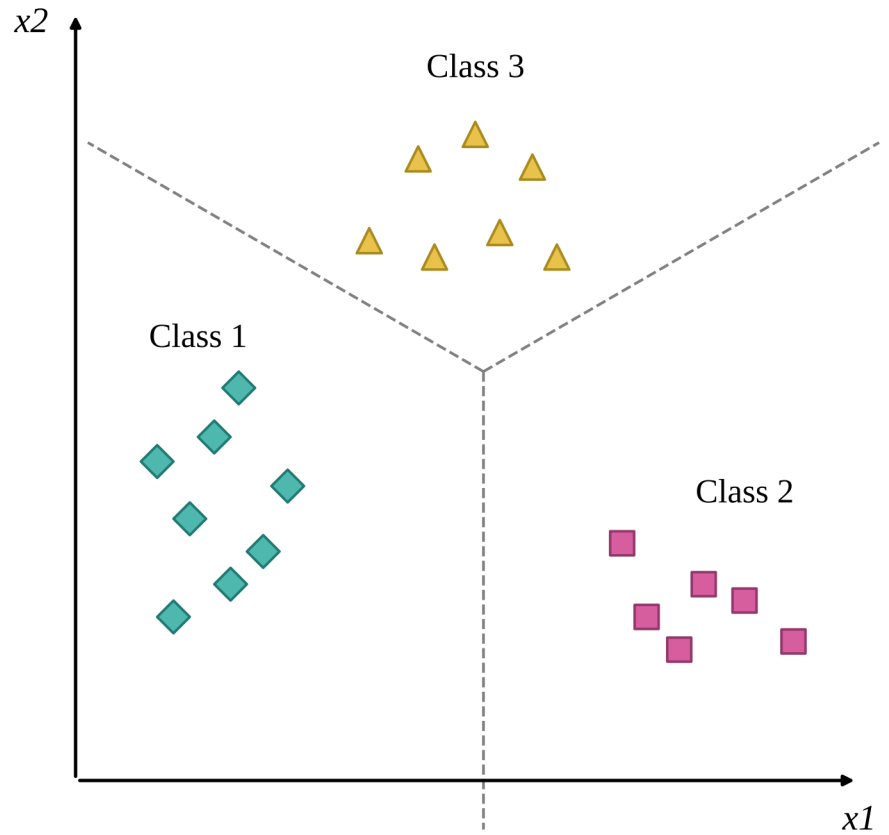
- Spam/Not Spam
- Positive/Negative/Neutral
- Cat/Dog/Bird/Mouse
- 1/2/3/4

Recap

❖ Getting Started



Binary Classification (n_classes=2)



Multi-class Classification (n_classes >= 2)

Recap

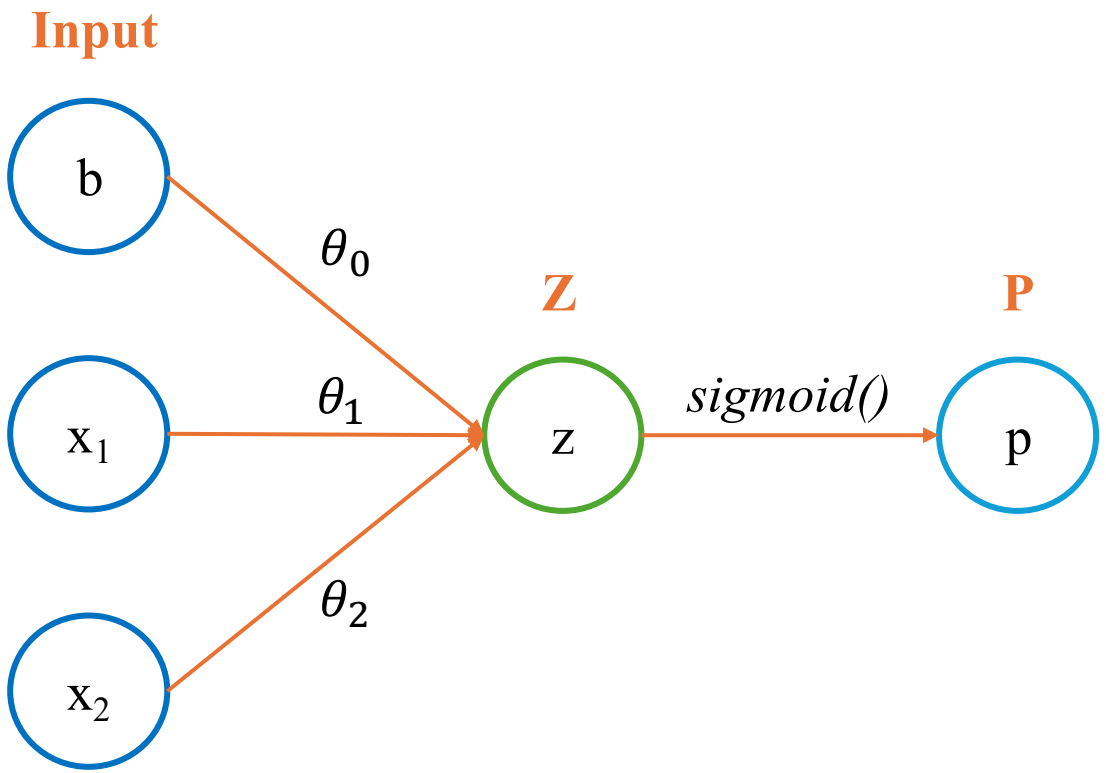
❖ Logistic Regression (Graph)

x : Input data x

z : Linear value (z)

p : Activation value (p)

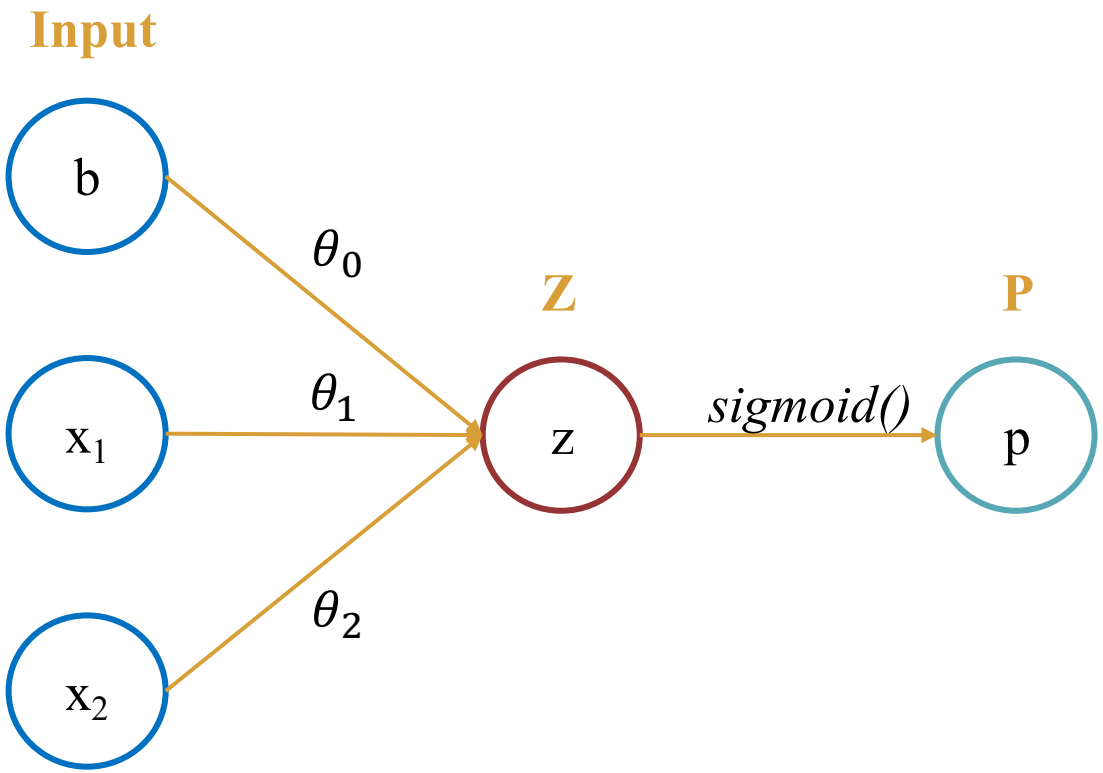
: Direction



Logistic Regression Computation

Recap

❖ Logistic Regression



z : Linear value

Green node represents the dot product of input features and their weights (including bias):

$$z = X \cdot \theta = \sum_{i=1}^{n} X_i \theta_i$$

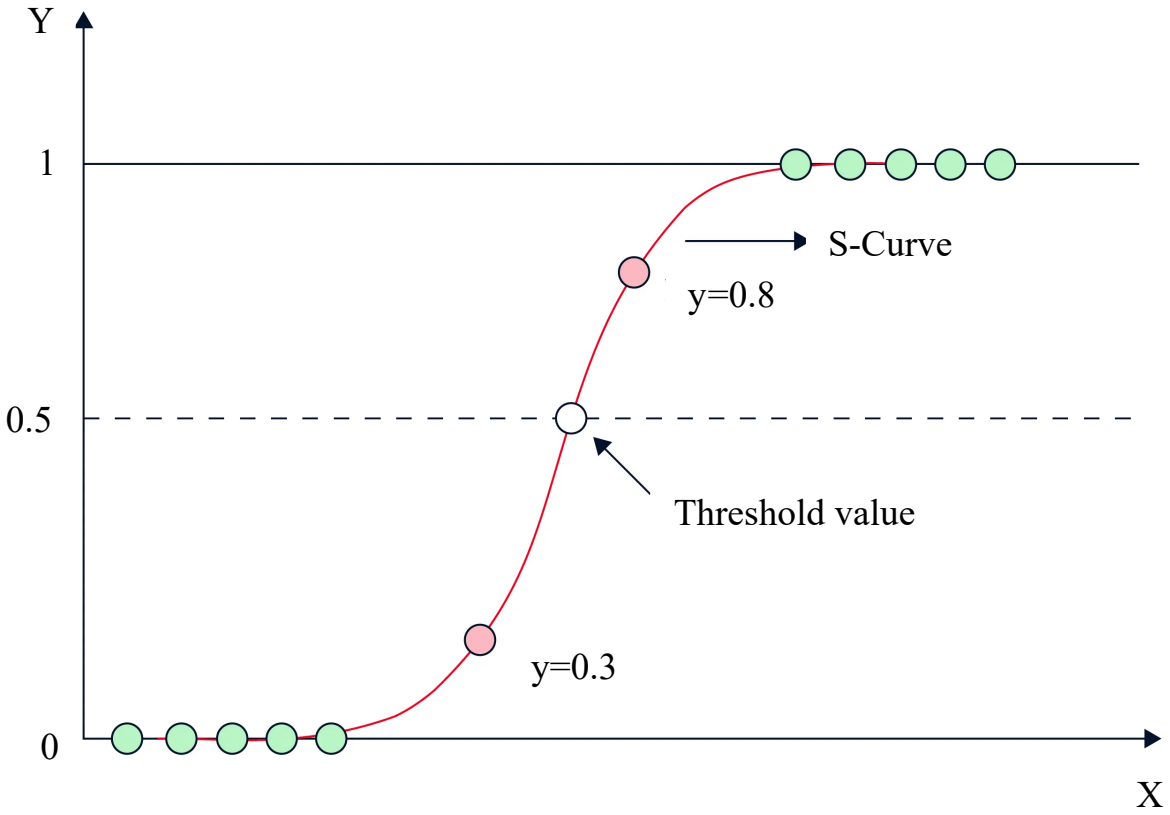
p : Activation value (p)

Yellow node represents the output of sigmoid function (activation function) with the input of z:

$$a(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Recap

❖ Logistic Regression for Multiclass?



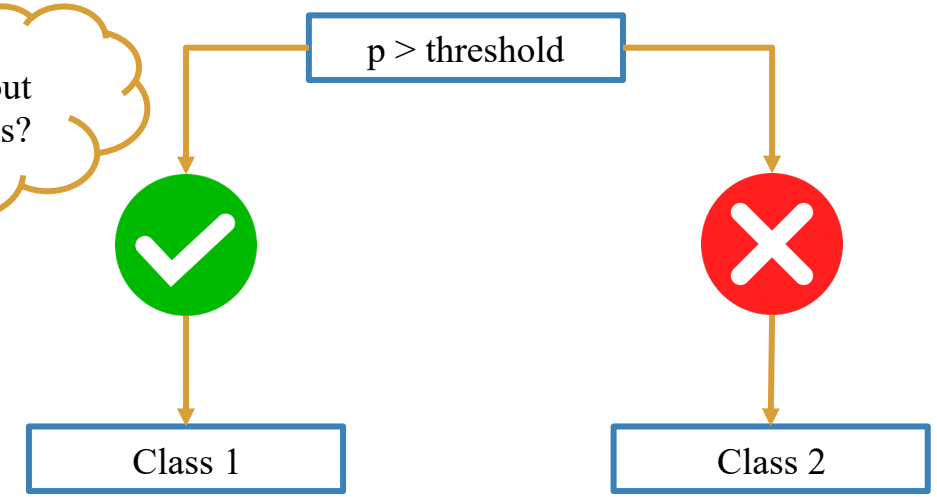
Sigmoid function

$$z = b_0 + \sum_{i=1}^{n_features} b_i x_i$$

$$sigmoid(z) = \frac{1}{1 + e^{-z}}$$

Range value: (0, 1)

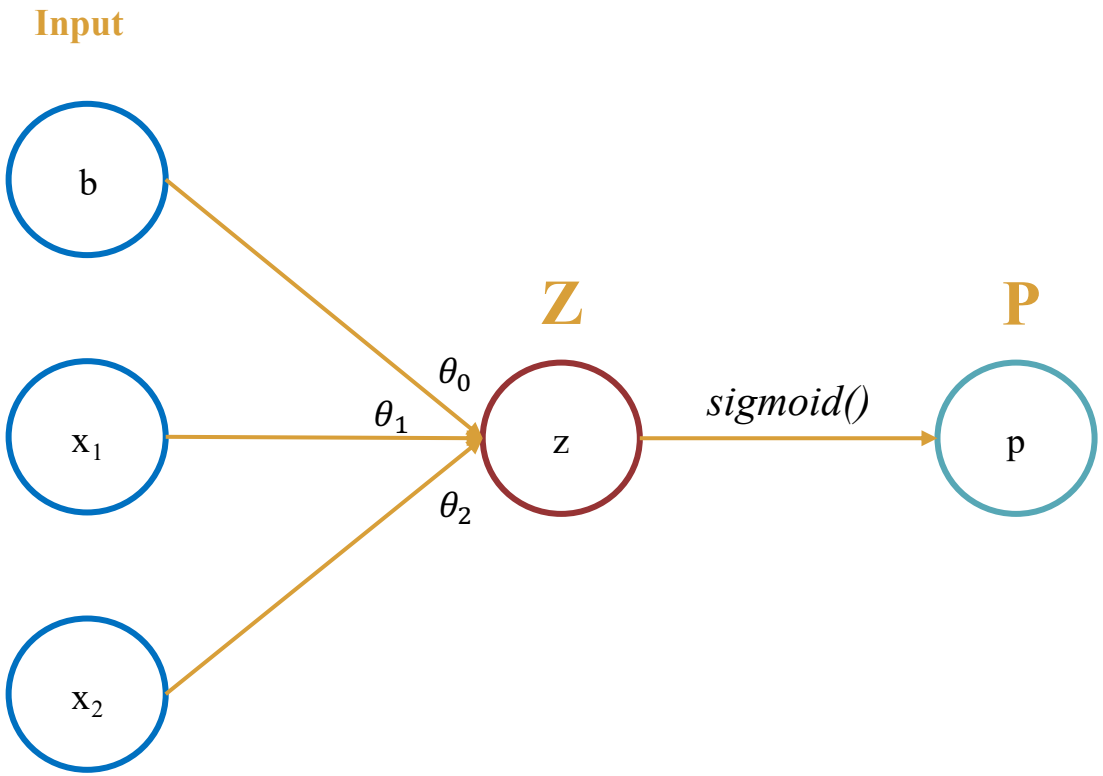
How about multiclass?



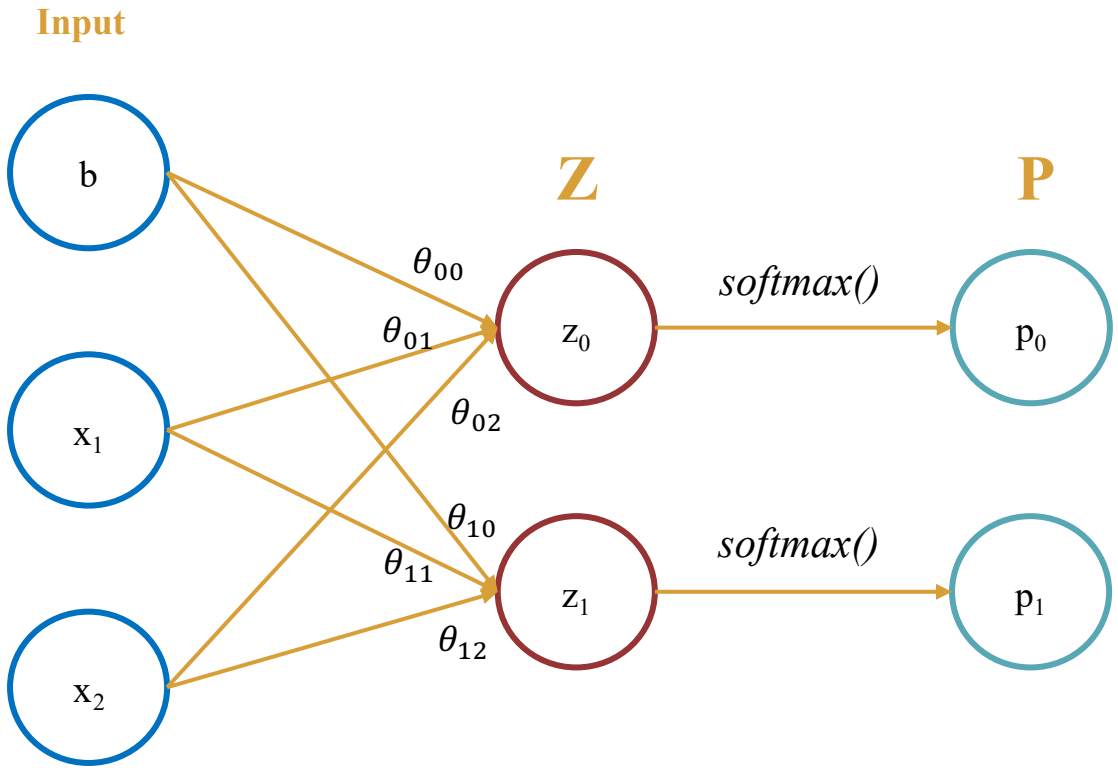
Recap

❖ Softmax Regression

n_features = 2, n_classes = 2



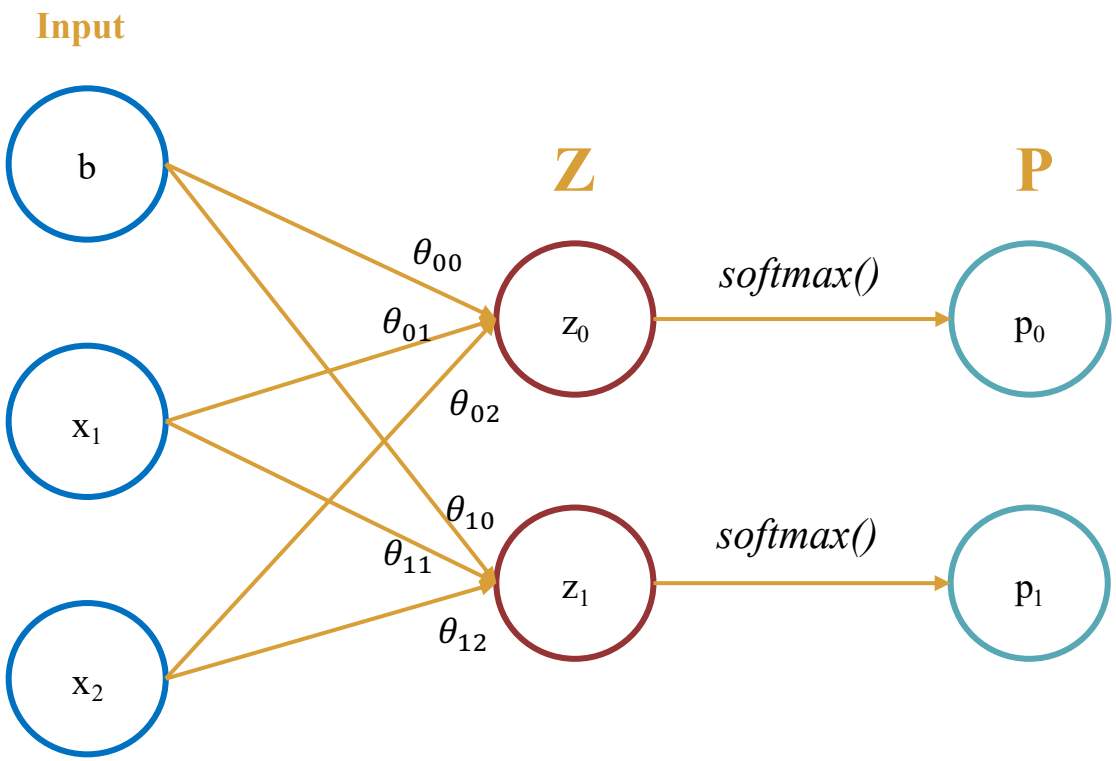
Logistic Regression



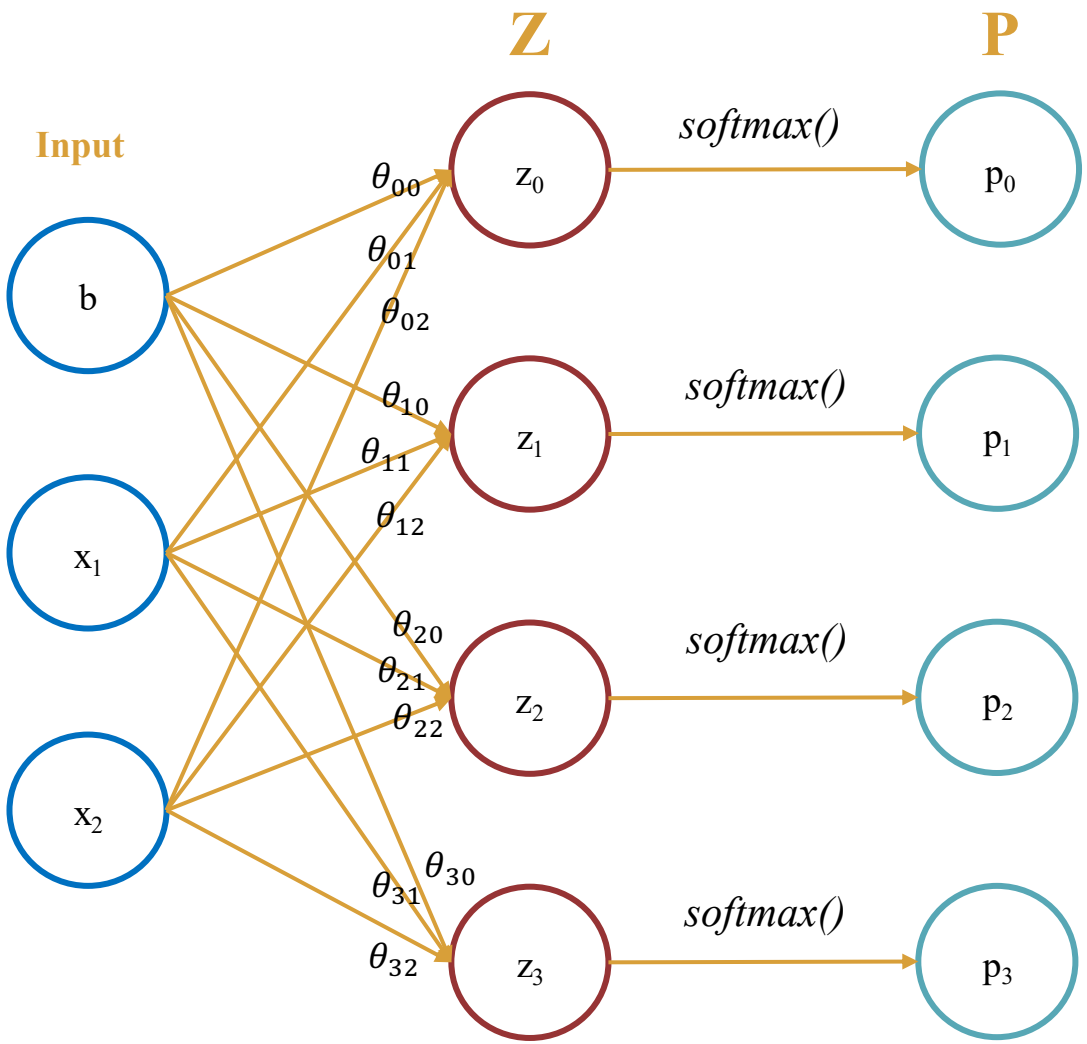
Softmax Regression

Recap

❖ Softmax Regression



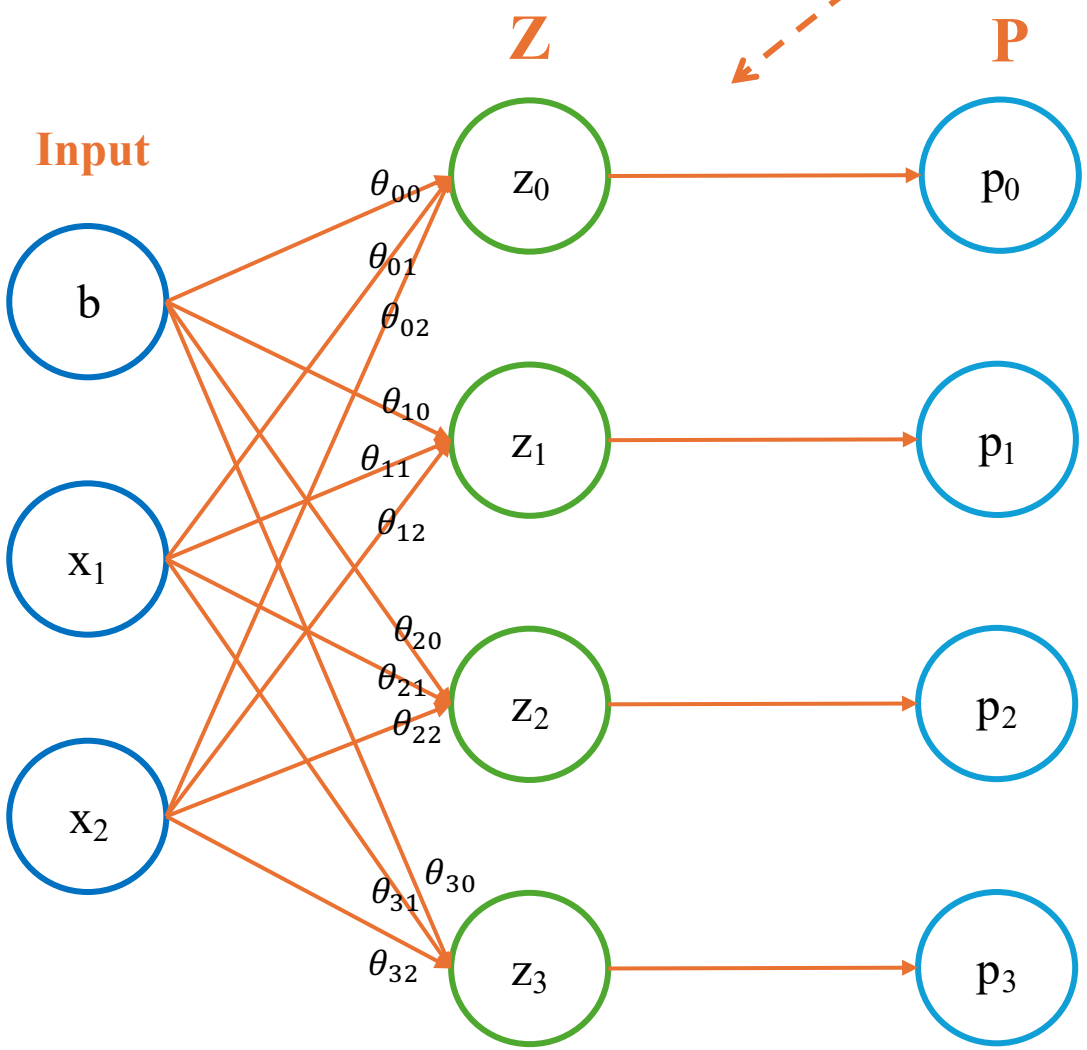
Softmax Regression (2 class)



Softmax Regression (4 class)

Recap

❖ Softmax Regression



n_features = 2, n_classes = 4

❖ Calculate z of class i-th:

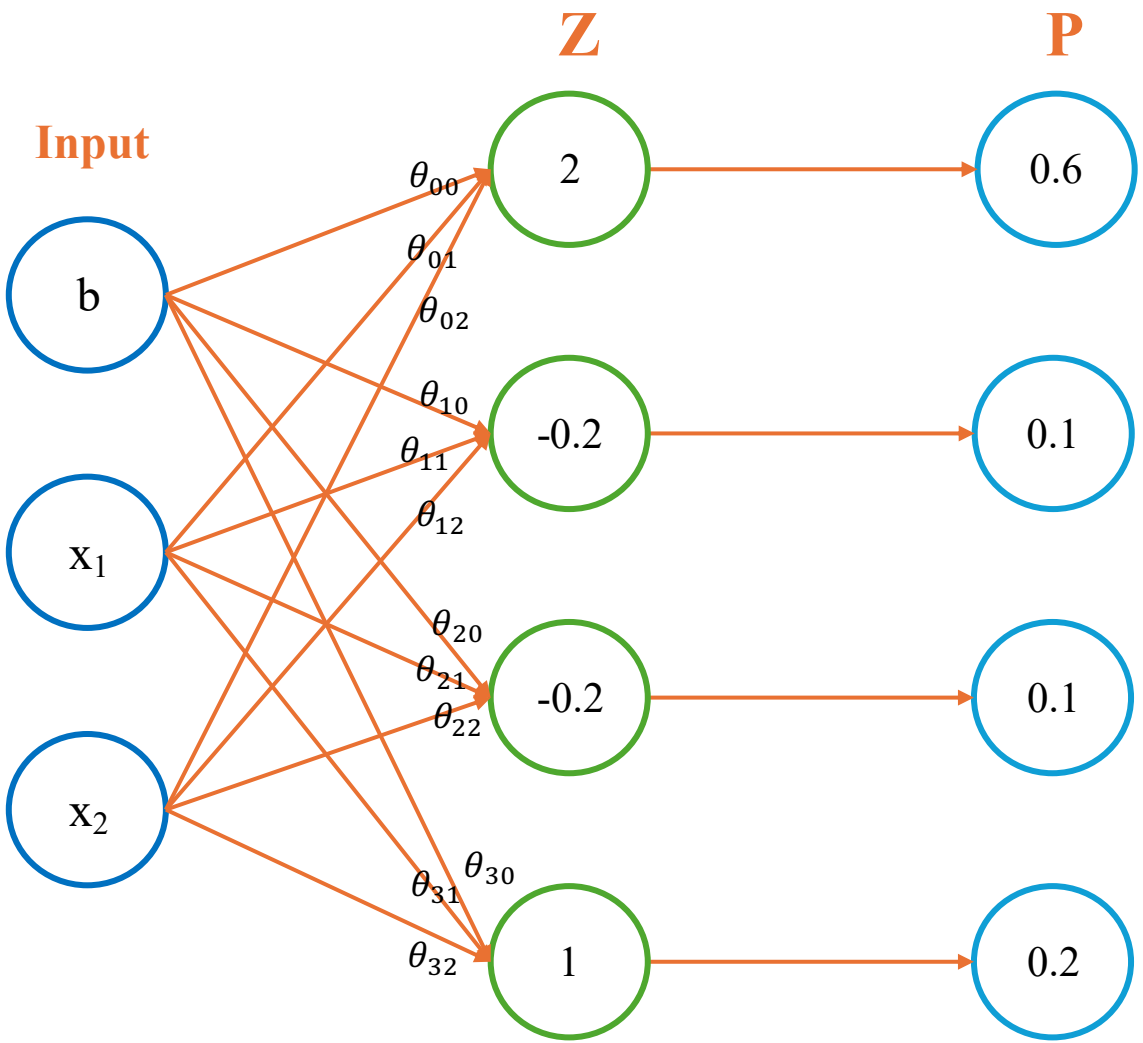
$$z_i = \theta_i^T X = b\theta_{i0} + \sum_{j=1}^{n_features} \theta_{ij} X_j$$

❖ Calculate p of class i-th:

$$p_i = softmax(Z) = \frac{e^{z_i}}{\sum_{j=1}^{n_classes} e^{z_j}}$$

Recap

❖ Softmax Regression



n_features = 2, n_classes = 4

$$1) 0 < p_i < 1$$

$$2) \sum_i^{n_classes} p_i = 1$$

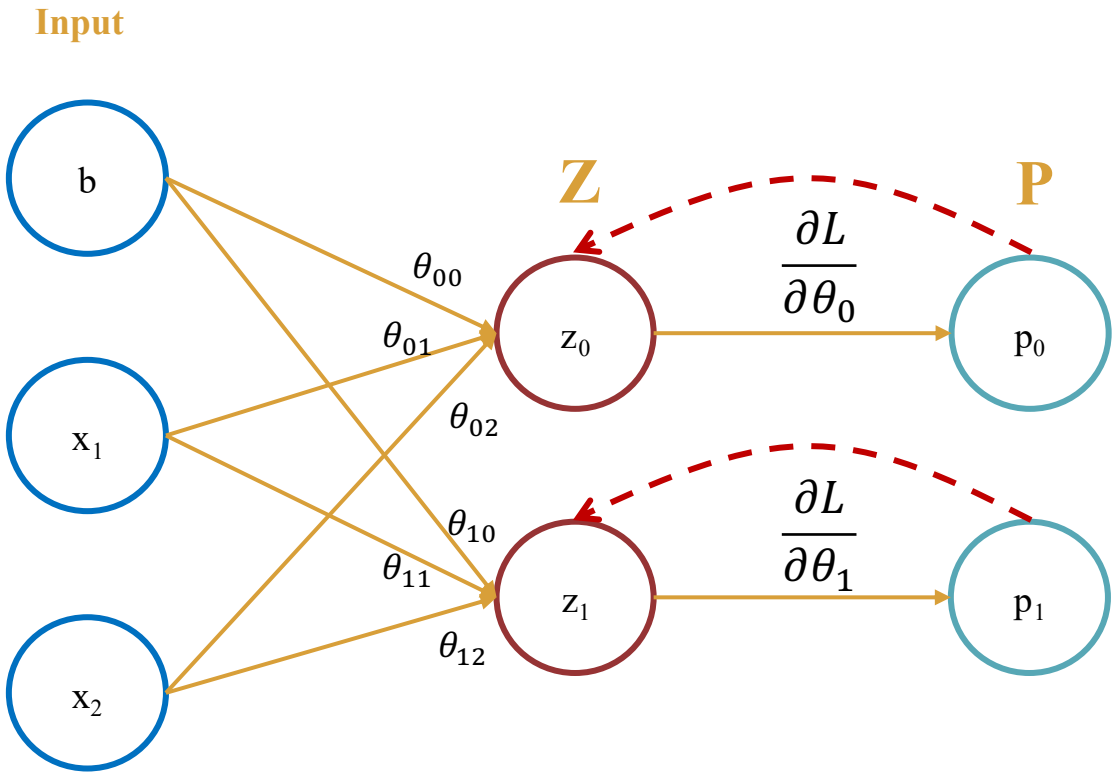
$$3) z_i < 0, p_i > 0$$

$$4) z_i = z_j, p_i = p_j$$

$$5) max = z_i, max = p_i$$

Recap

❖ Softmax Regression



❖ Cross-entropy loss:

$$L(\hat{y}, y) = -\log(y^T \cdot \hat{y}) = -y^T \cdot \log(\hat{y})$$

❖ Derivative:

$$\nabla_\theta L = \frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial \theta} = X \cdot (\hat{y} - y)^T$$

Recap

❖ Softmax Regression

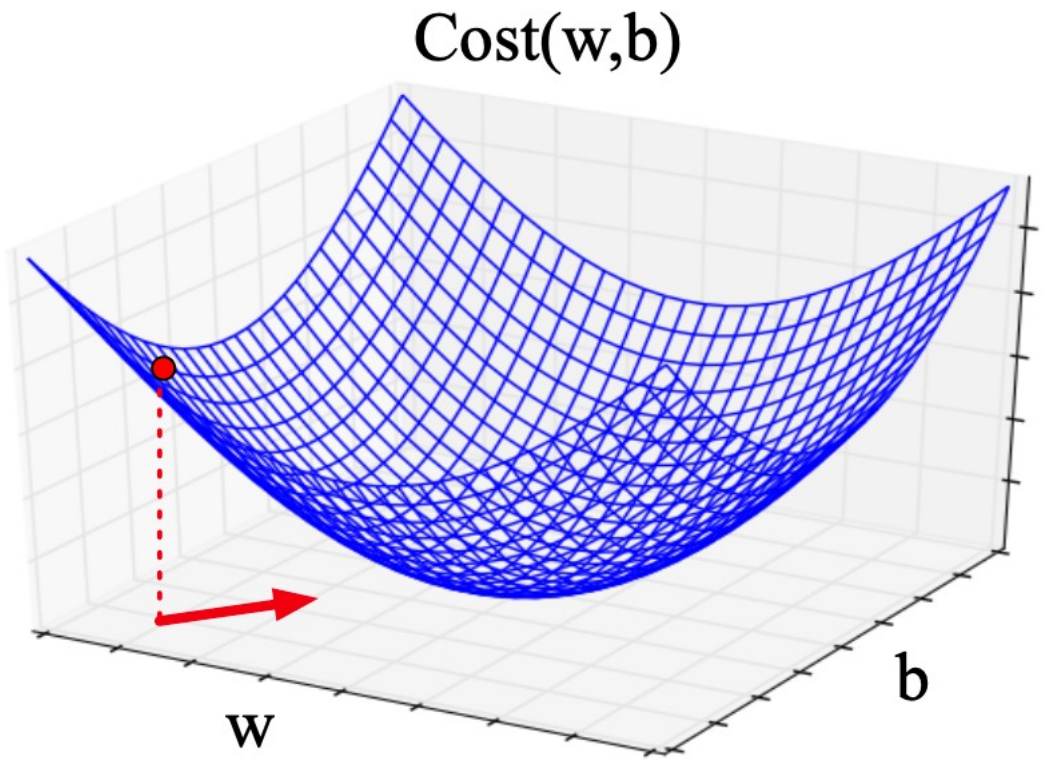
Our objective is to find the optimal weights and bias (theta) that minimize the loss function:

$$\hat{\theta} = arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} L(\hat{y}^i, y^i)$$

For each step, compute the gradient and update the current theta:

$$\nabla_{\theta_s} L = \frac{1}{N} X^{i^T} (\hat{y}^i - y^i)$$

$$\theta_{s+1} = \theta_s - \eta \nabla_{\theta_s} L$$



Gradient Descent

Recap

❖ Softmax Regression Using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\boldsymbol{z} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$\boldsymbol{d} = [1 \ \ldots 1]\boldsymbol{e^z}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{e^z} \emptyset \boldsymbol{d}$$

$\emptyset$ is Hadamard division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\boldsymbol{y}^T log\hat{\boldsymbol{y}}$$

4) Compute derivative

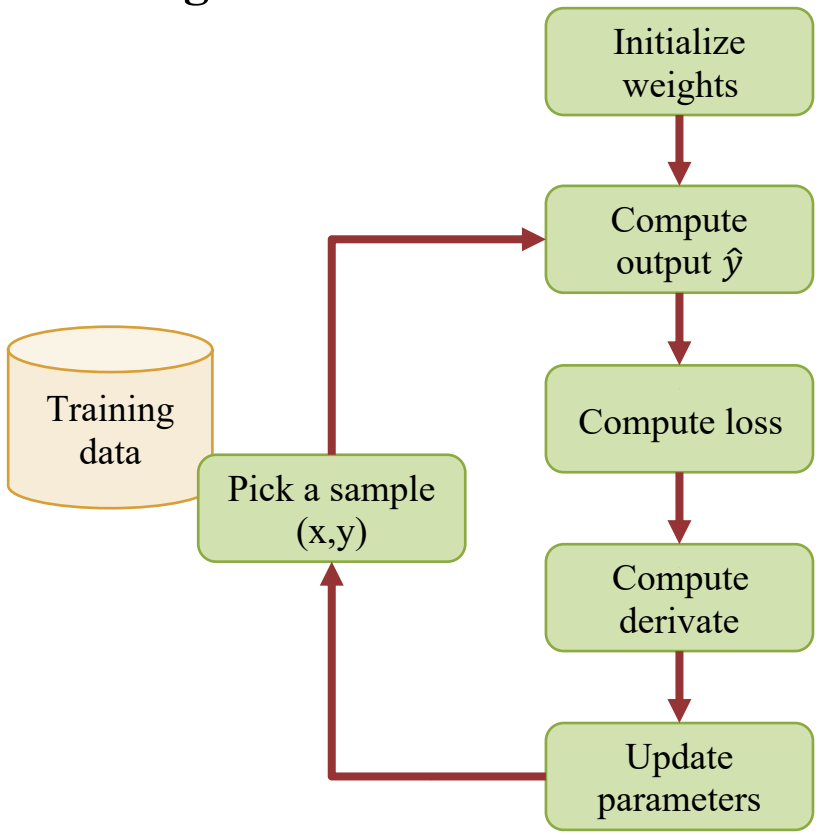
$$\nabla_\theta \mathrm{L} = \boldsymbol{x}(\hat{\boldsymbol{y}} - \boldsymbol{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_\theta \mathrm{L}$$

η is learning rate

Training Phase



Recap

❖ Softmax Regression Using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\boldsymbol{z} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$\boldsymbol{d} = [1 \ \ldots 1]\boldsymbol{e^z}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{e^z} \emptyset \boldsymbol{d}$$

$\emptyset$ is Hadamard division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\boldsymbol{y}^T log \hat{\boldsymbol{y}}$$

4) Compute derivative

$$\nabla_\theta \mathrm{L} = \boldsymbol{x}(\hat{\boldsymbol{y}} - \boldsymbol{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_\theta \mathrm{L}$$

η is learning rate

$$\boldsymbol{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

$$\boldsymbol{y} = [0]$$

One-hot encoding for label

$$\boldsymbol{y} = 0 \rightarrow \boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\boldsymbol{y} = 1 \rightarrow \boldsymbol{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} \mathrm{b}_0 & \mathrm{b}_1 \\ \mathrm{w}_0 & \mathrm{w}_1 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

$$\boldsymbol{\eta} = 0.1$$

Hours	Pass
0.5	0
1.0	0
1.5	1
2.0	1

Recap

❖ Softmax Regression Using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\boldsymbol{z} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$\boldsymbol{d} = [1 \ \ldots 1]\boldsymbol{e^z}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{e^z} \emptyset \boldsymbol{d}$$

$\emptyset$ is Hadamard division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\boldsymbol{y}^T log \hat{\boldsymbol{y}}$$

4) Compute derivative

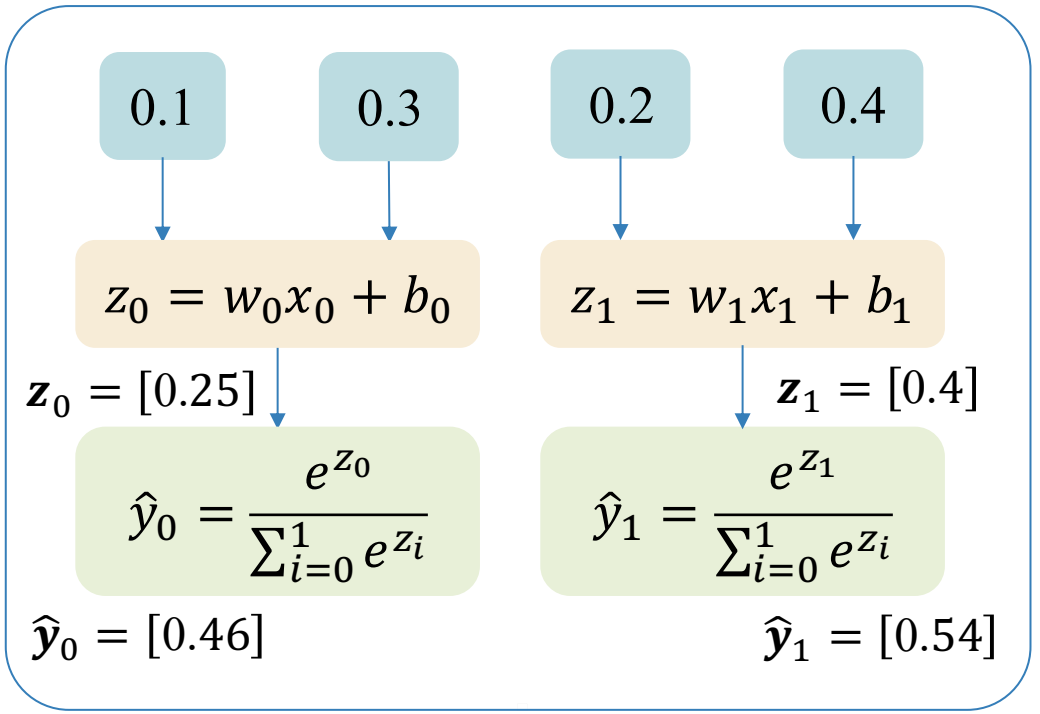
$$\nabla_\theta \mathrm{L} = \boldsymbol{x}(\hat{\boldsymbol{y}} - \boldsymbol{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_\theta \mathrm{L}$$

η is learning rate

$$\boldsymbol{\eta} = 0.01 \qquad \boldsymbol{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \qquad \boldsymbol{y} = 0 \rightarrow \boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



Recap

❖ Softmax Regression Using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\boldsymbol{z} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$\boldsymbol{d} = [1 \ \ldots 1]\boldsymbol{e^z}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{e^z} \emptyset \boldsymbol{d}$$

$\emptyset$ is Hadamard division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\boldsymbol{y}^T log\hat{\boldsymbol{y}}$$

4) Compute derivative

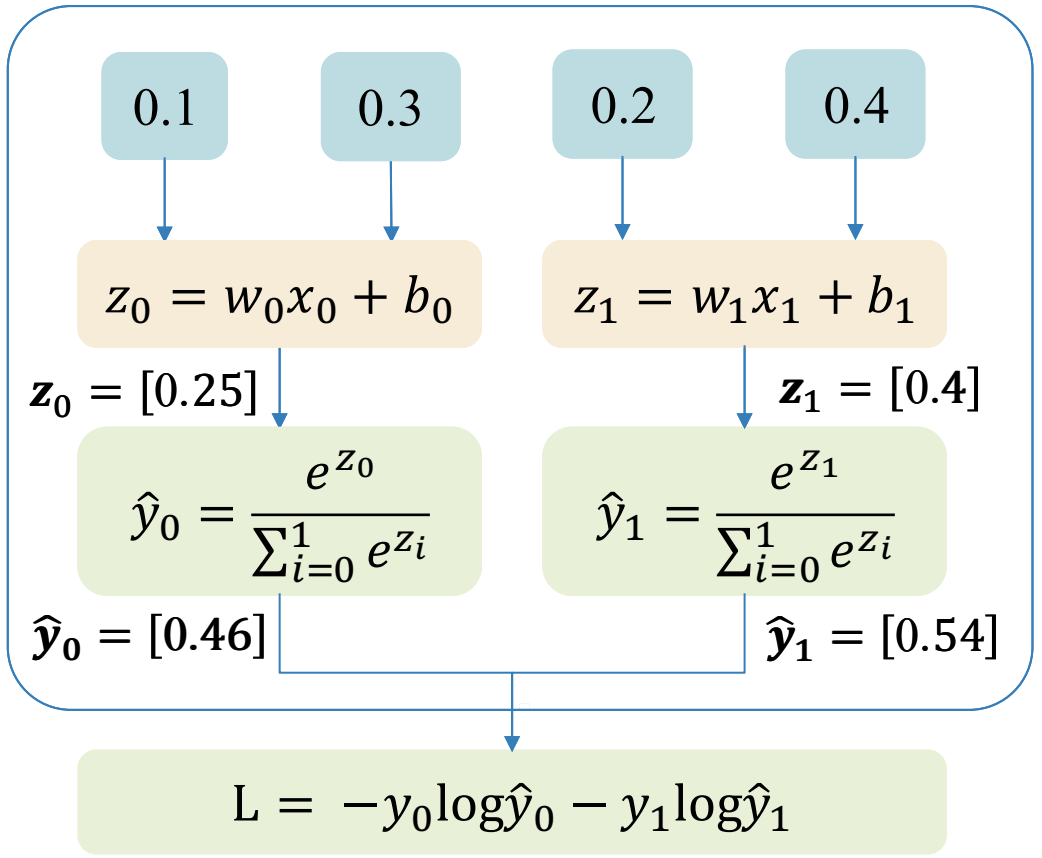
$$\nabla_\theta \mathrm{L} = \boldsymbol{x}(\hat{\boldsymbol{y}} - \boldsymbol{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_\theta \mathrm{L}$$

η is learning rate

$$\boldsymbol{\eta} = 0.01 \qquad \boldsymbol{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \qquad \boldsymbol{y} = 0 \rightarrow \boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



Recap

❖ Softmax Regression Using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\boldsymbol{z} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$\boldsymbol{d} = [1 \ \ldots 1]\boldsymbol{e^z}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{e^z} \emptyset \boldsymbol{d}$$

$\emptyset$ is Hadamard division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\boldsymbol{y}^T log\hat{\boldsymbol{y}}$$

4) Compute derivative

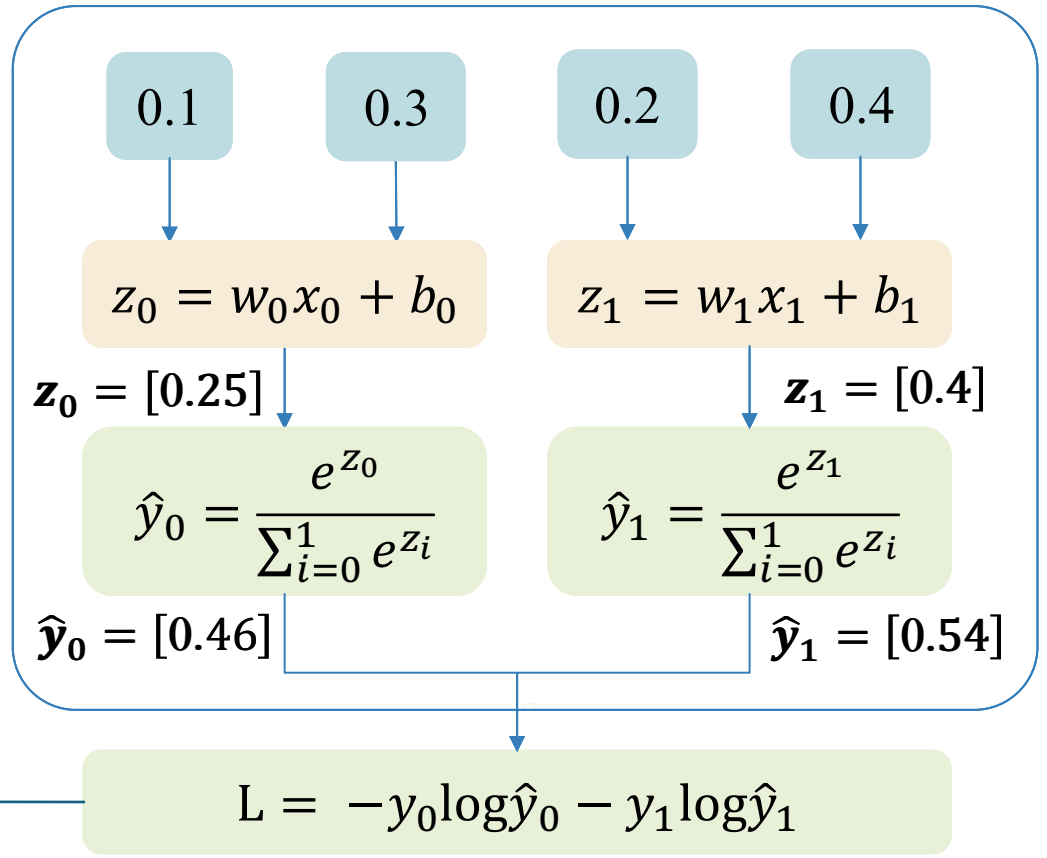
$$\nabla_\theta \mathrm{L} = \boldsymbol{x}(\hat{\boldsymbol{y}} - \boldsymbol{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_\theta \mathrm{L}$$

η is learning rate

$$\boldsymbol{\eta} = 0.01 \qquad \boldsymbol{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \qquad \boldsymbol{y} = 0 \rightarrow \boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$



Recap

❖ Softmax Regression Using Gradient Descent

1) Pick a sample from training data

2) Compute output $\hat{y}$

$$\boldsymbol{z} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$\boldsymbol{d} = [1 \ \ldots 1]\boldsymbol{e^z}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{e^z} \emptyset \boldsymbol{d}$$

$\emptyset$ is Hadamard division

3) Compute loss (cross-entropy)

$$L(\boldsymbol{\theta}) = -\boldsymbol{y}^T log\hat{\boldsymbol{y}}$$

4) Compute derivative

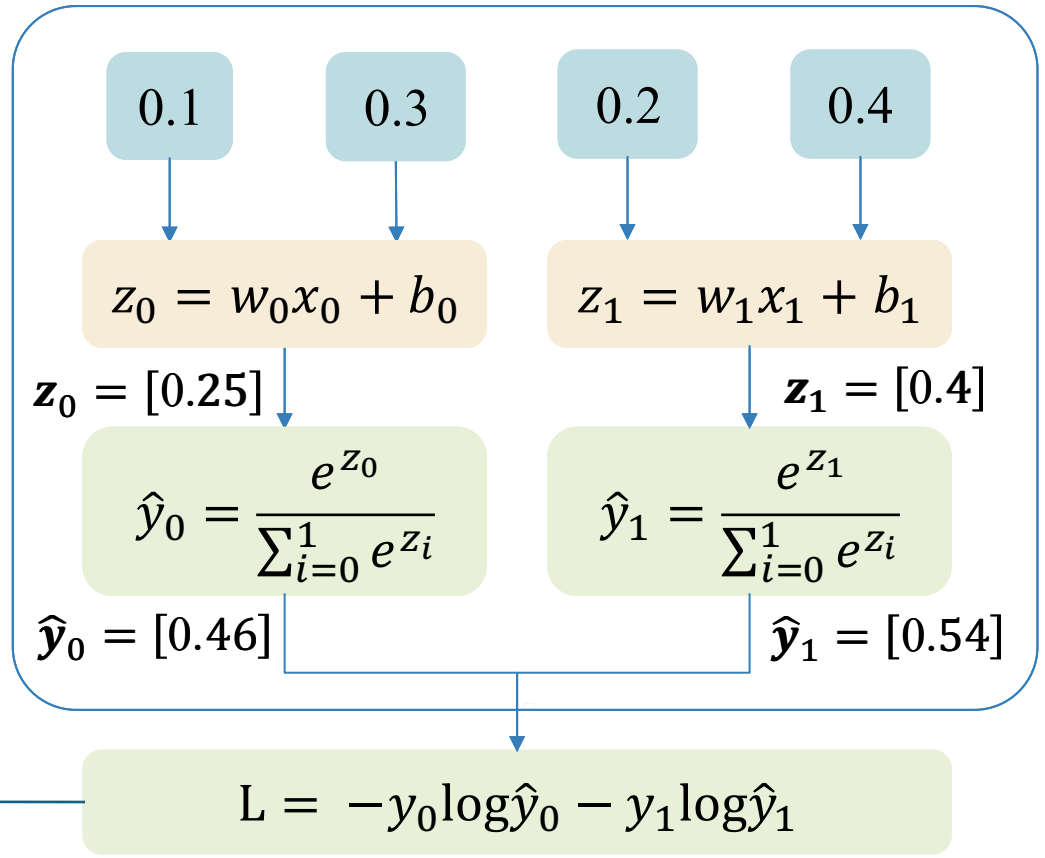
$$\nabla_\theta \mathrm{L} = \boldsymbol{x}(\hat{\boldsymbol{y}} - \boldsymbol{y})^T$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_\theta \mathrm{L}$$

η is learning rate

$$\eta = 0.01 \qquad \boldsymbol{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \qquad \boldsymbol{y} = 0 \rightarrow \boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$

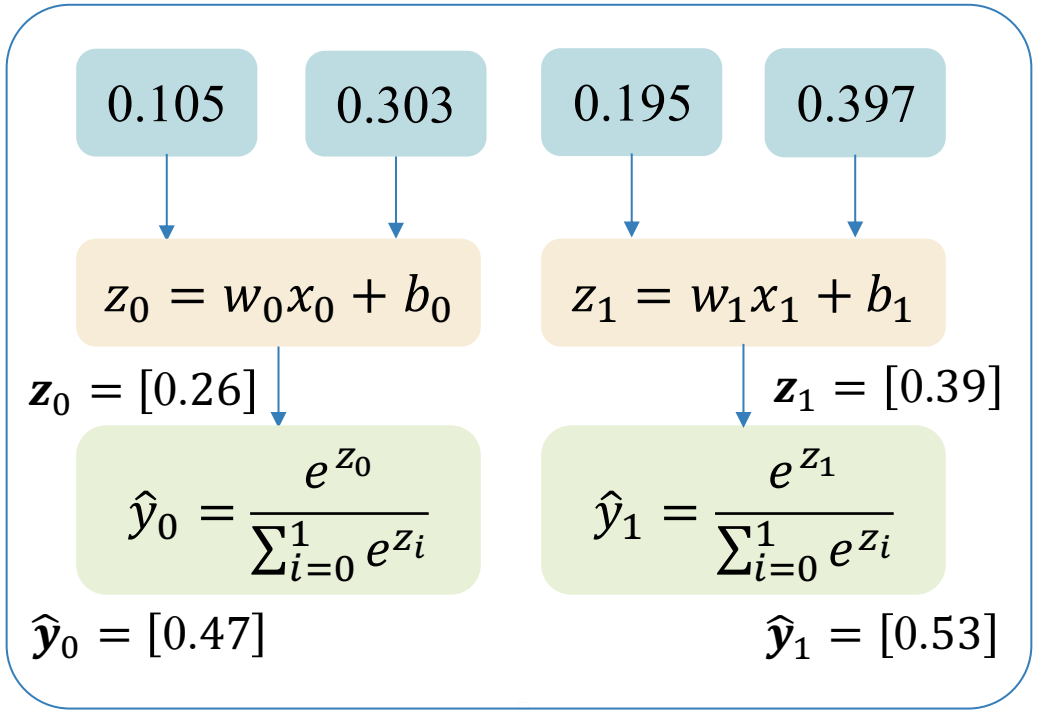


Recap

❖ Softmax Regression Using Gradient Descent

Testing Phase

Hours	Pass
0.5	?



$y_{pred} = 1$

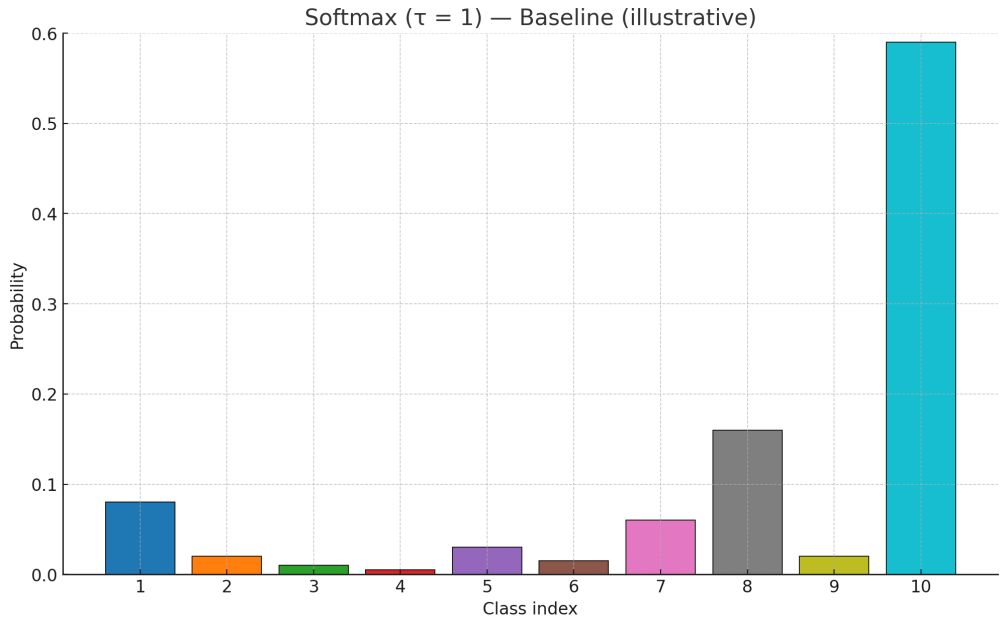
Recap

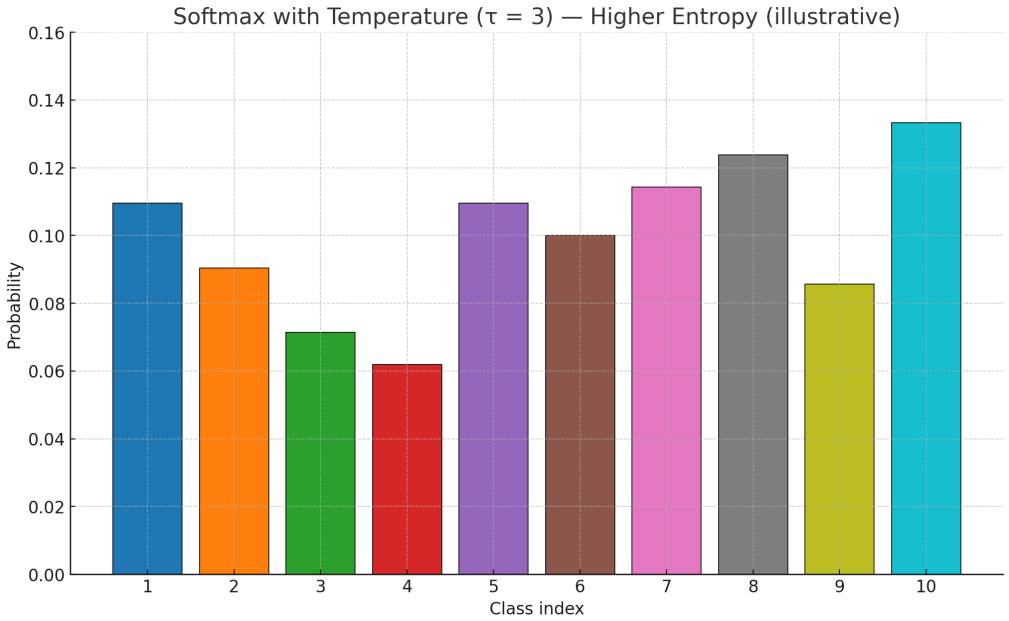
❖ Softmax Regression With Temperature

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

Softmax with temperature

$$p_i = \frac{e^{z_i/\tau}}{\sum_{j=1}^{N} e^{z_j/\tau}}$$





Recap

❖ Softmax Regression With Numerical Stability

Softmax Can Be Tricky to Compute!

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

Can overflow!

Exponentials are hard to compute!

$e^{89} \approx 4.4 \times 10^{38} > 32\text{-bit float limit: } 3.4 \times 10^{38}$

=> Exponential can float-overflow!

Problem: Compute the maximum value.

$$m = \max\{z_1, z_2, \ldots, z_N\}$$

And modify the original formula

$$Softmax(z_i) = \frac{e^{z_i - m}}{\sum_{j=1}^{N} e^{z_j - m}}$$

Never overflow!

$$\frac{e^{z_i - m}}{\sum_{j=1}^{N} e^{z_j - m}} = \frac{e^{-m} e^{z_i}}{e^{-m} \sum_{j=1}^{N} e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

=> This is mathematically equivalent but numerically stable!

https://github.com/scipy/scipy/blob/v1.11.1/scipy/special/_logsumexp.py
https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/utils/extmath.py

Recap

❖ QUIZ

Q1. Which function converts a vector of raw scores (logits) into class probabilities that sum to 1?
A. Sigmoid function
B. Softmax function
C. ReLU function
D. Cross-entropy loss

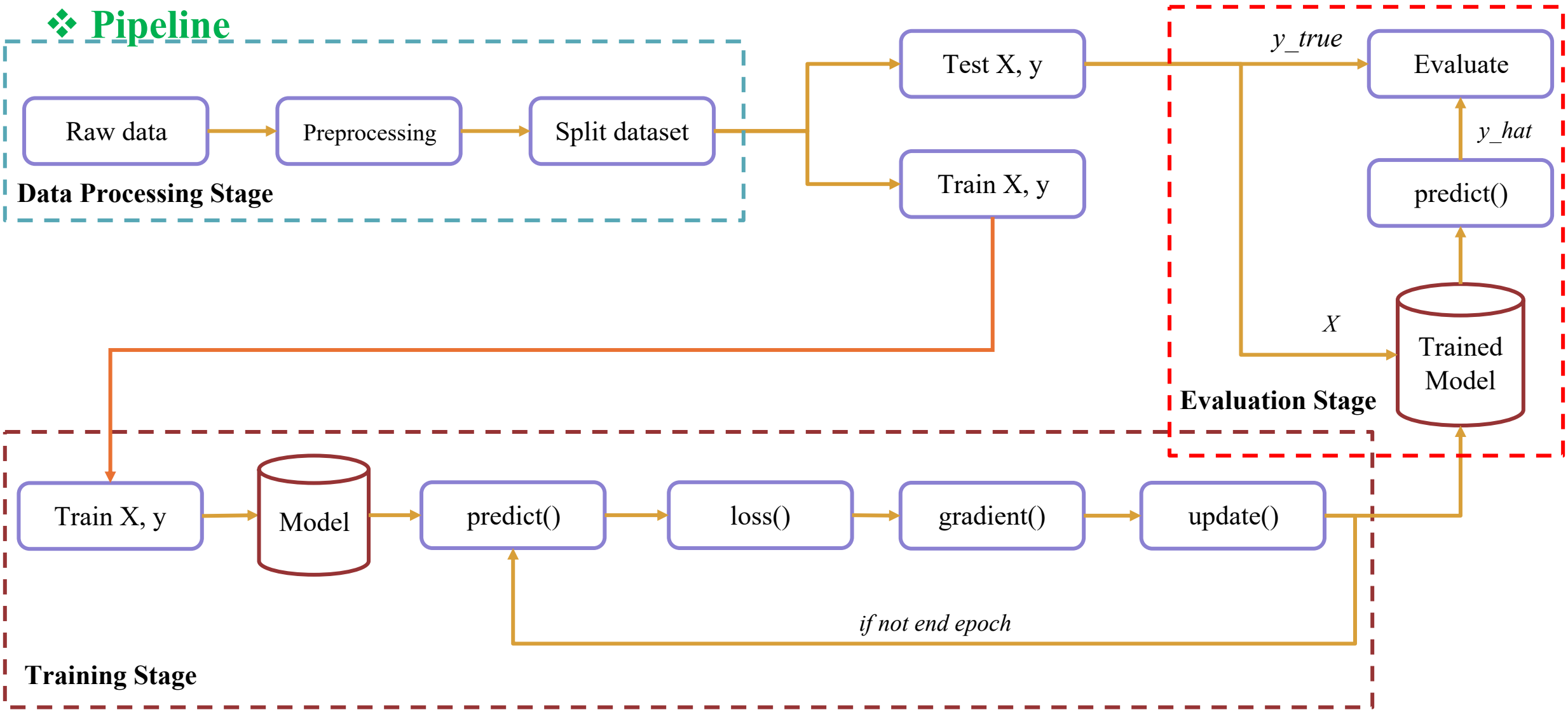
Q2. Which loss function is used to train Softmax Regression?
A. Mean Squared Error
B. Hinge Loss
C. Cross-Entropy Loss
D. L1 Loss

Q3. Why do we subtract max(z) before computing softmax in practice?
A. To change the final probabilities
B. To avoid numerical overflow from large exponentials
C. To speed up training only
D. To normalize the input features

Exercise 1

Exercise 1

❖ Pipeline



Exercise 1

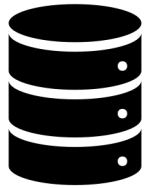
❖ Card Fraud Detection

➢ Given card_fraud_detection.csv dataset, build a Card Fraud Detection model using Softmax Regression.



Exercise 1

❖ Step 1: Load Dataset



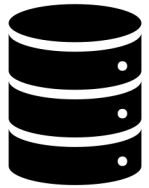
Dataset

```
1 dataset_path = 'creditcard.csv'
2 df = pd.read_csv(
3     dataset_path
4 )
5 df
```

	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

Exercise 1

❖ Step 1: Load Dataset



Dataset

```
1 df.describe()
```

	Time	V1	V2	V3	V4	V5	V6
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.604066e-16	1.487313e-15
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01

8 rows × 31 columns

Exercise 1

❖ Step 2: Get Inputs And Labels

Input

Labels

Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.918215	7.305334	1.914428	...	0.213454	0.111864	1.014480	-0.509348	1.436807	0.250034	0.943651	0.823731	0.77	0
787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.024330	0.294869	0.584800	...	0.214205	0.924384	0.012463	-1.016226	-0.606624	-0.395255	0.068472	-0.053527	24.79	0
788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.296827	0.708417	0.432454	...	0.232045	0.578229	-0.037501	0.640134	0.265745	-0.087371	0.004455	-0.026561	67.88	0
788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.686180	0.679145	0.392087	...	0.265245	0.800049	-0.163298	0.123205	-0.569159	0.546668	0.108821	0.104533	10.00	0
792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.577006	-0.414650	0.486180	...	0.261057	0.643078	0.376777	0.008797	-0.473649	-0.818267	-0.002415	0.013649	217.00	0

Exercise 1

❖ Step 2: Get Inputs And Labels



Pre-processed Dataset

```
1 dataset_arr = df.to_numpy()
2 X = dataset_arr[:, :-1].astype(np.float64)
3 y = dataset_arr[:, -1].astype(np.uint8)
```

```
1 dataset_arr = df.to_numpy()
```

```
1 print(X.shape)
```

```
(284807, 30)
```

Exercise 1

❖ Step 3: Add the Bias Term

❖ Calculate z of class i'th:

$$z_i = \theta_i^T X = b\theta_{i0} + \sum_{j=1}^{n_features} \theta_{ij} X_j$$

1. Ones Vector (n_elements = n_samples).

idx	0	1	…	n_samples - 1
value	1	1	1	1

2. Concat by column to independent variable.

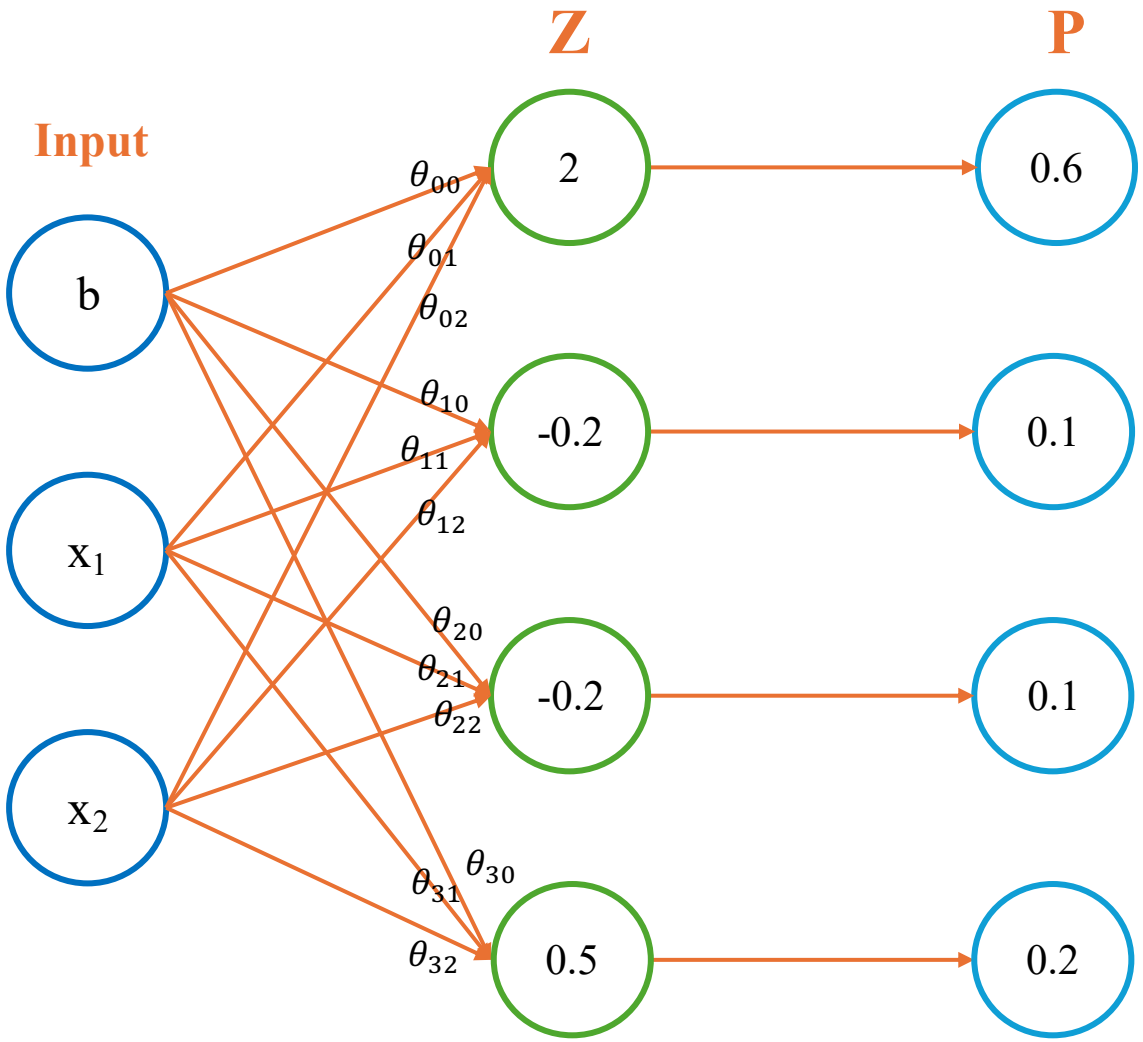
Bias
1
1
1
1

Feature 1	Feature 2	Feature 3	Feature 4
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…

```
intercept = np.ones((
    X.shape[0], 1)
)
X_b = np.concatenate(
    (intercept, X),
    axis=1
)
```

Exercise 1

❖ Step 4: One-hot Encoding Label



❖ **Recall:** Predict on sample i'th output an n elements vector

$y_hat_i = [0.6, 0.1, 0.1, 0.2]$

$y_i = 0$

```
1 y
```

```
array([0., 0., 0., ..., 0., 0., 0.])
```

How to calculate loss between y vs y_hat?

One hot encoding label

Exercise 1

❖ Step 4: One-hot Encoding Label

➢ **Definition:** Convert categorial data to one-hot vector.

n_features = 2, n_classes = 4

1. Zeros Vector (n_elements = n_classes)

idx	0	1	2	3
value	0	0	0	0

2. E.g: y = 2

zeros_vector[y] = 1

idx	0	1	2	3
value	0	0	1	0

Label encoding

Label
0
0
1
3
2
1
2

One-hot encoding

Label			
1	0	0	0
1	0	0	0
0	1	0	0
0	0	0	1
0	0	1	0
0	1	0	0
0	0	1	0

Exercise 1

❖ Step 4: One-hot Encoding Label

➢ **Definition:** Convert categorial data to one-hot vector.

1. Zeros Vector (n_elements = n_classes)

idx	0	1	2	3
value	0	0	0	0

2. E.g: y = 2

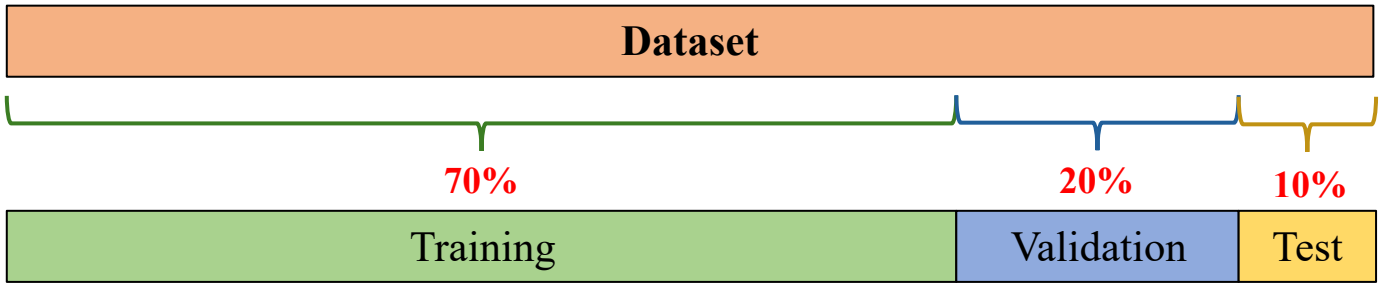
zeros_vector[y] = 1

idx	0	1	2	3
value	0	0	1	0

```
1 n_classes = np.unique(y, axis=0).shape[0]
2 n_samples = y.shape[0]
3
4 y_encoded = np.array(
5     [np.zeros(n_classes) for _ in range(n_samples)]
6 )
7 y_encoded[np.arange(n_samples), y] = 1
```

Exercise 1

❖ Step 5: Split Train, Val, Test Set




```python
val_size = 0.2
test_size = 0.125
random_state = 2
is_shuffle = True

X_train, X_val, y_train, y_val = train_test_split(
    X_b, y_encoded,
    test_size=val_size,
    random_state=random_state,
    shuffle=is_shuffle
)
```

```python
X_train, X_test, y_train, y_test = train_test_split(
    X_train, y_train,
    test_size=test_size,
    random_state=random_state,
    shuffle=is_shuffle
)
```

Exercise 1

❖ Step 6: Normalization

Using sklearn.preprocessing.StandardScaler() to scale all values in dataset.

$$z = \frac{x_i - \mu}{\sigma}$$

Note: We only use the train set to fit the scaler.

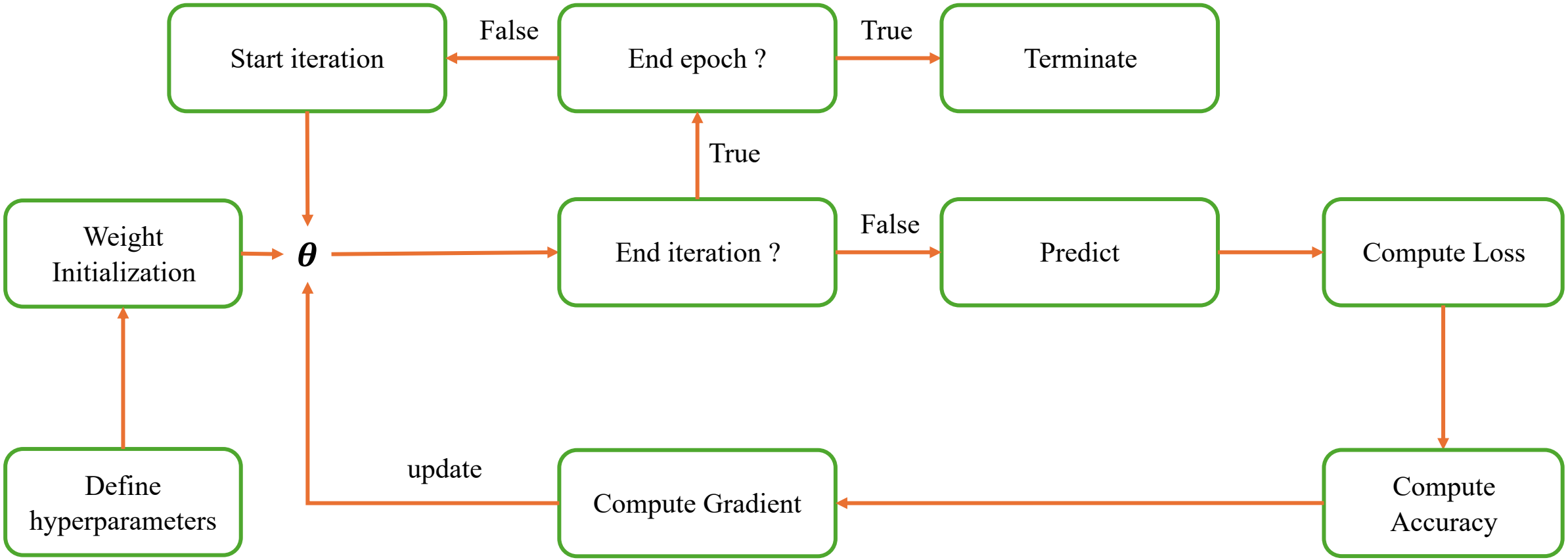
```
1 normalizer = StandardScaler()
2 X_train[:, 1:] = normalizer.fit_transform(X_train[:, 1:])
3 X_val[:, 1:] = normalizer.transform(X_val[:, 1:])
4 X_test[:, 1:] = normalizer.transform(X_test[:, 1:])
```

```
1 X_train
```

```
array([[ 1.        , -0.68101315,  0.71447649, ...,  0.1019486 ,
         0.07410031, -0.29192049],
       [ 1.        ,  1.04720051,  1.0475128 , ..., -0.19095883,
        -0.09813199, -0.27780616],
       [ 1.        ,  1.54445364, -1.20352757, ...,  0.31391593,
         1.12488897, -0.34496952],
       ...,
       [ 1.        ,  0.86681249, -1.80351772, ..., -2.8881391 ,
         1.19656217,  0.00738136],
       [ 1.        ,  1.14262225, -0.53304231, ...,  1.57192248,
         1.15765039, -0.00520879],
       [ 1.        , -0.89535081, -0.86109348, ...,  0.92494536,
         0.36676226,  0.26471862]])
```

Exercise 1

❖ Step 7: Training



Exercise 1

❖ Step 7: Training

```python
def softmax(z):
    exp_z = np.exp(z)

    return exp_z / exp_z.sum(axis=1)[:, None]
```

```python
def compute_loss(y_hat, y):
    n = y.size

    return (-1 / n) * np.sum(y * np.log(y_hat))
```

```python
def predict(X, theta):
    z = np.dot(X, theta)
    y_hat = softmax(z)

    return y_hat
```

```python
def compute_gradient(X, y, y_hat):
    n = y.size

    return np.dot(X.T, (y_hat - y)) / n

def update_theta(theta, gradient, lr):
    return theta - lr * gradient
```

Exercise 1

❖ Step 7: Training

EPOCH 1:	Training loss: 0.189	Validation loss: 0.266
EPOCH 2:	Training loss: 0.133	Validation loss: 0.178
EPOCH 3:	Training loss: 0.106	Validation loss: 0.134
EPOCH 4:	Training loss: 0.092	Validation loss: 0.110
EPOCH 5:	Training loss: 0.084	Validation loss: 0.096
EPOCH 6:	Training loss: 0.079	Validation loss: 0.088
EPOCH 7:	Training loss: 0.076	Validation loss: 0.082
EPOCH 8:	Training loss: 0.074	Validation loss: 0.078
EPOCH 9:	Training loss: 0.073	Validation loss: 0.075
EPOCH 10:	Training loss: 0.071	Validation loss: 0.072

```
for i in range(0, X_train.shape[0], batch_size):
    X_i = X_train[i:i+batch_size]
    y_i = y_train[i:i+batch_size]

    y_hat = predict(X_i, theta)
    train_loss = compute_loss(y_hat, y_i)
    gradient = compute_gradient(X_i, y_i, y_hat)
    theta = update_theta(theta, gradient, lr)

    train_batch_losses.append(train_loss)
    train_acc = compute_accuracy(X_train, y_train, theta)
    train_batch_accs.append(train_acc)

    y_val_hat = predict(X_val, theta)
    val_loss = compute_loss(y_val_hat, y_val)
    val_batch_losses.append(val_loss)
```

Exercise 1

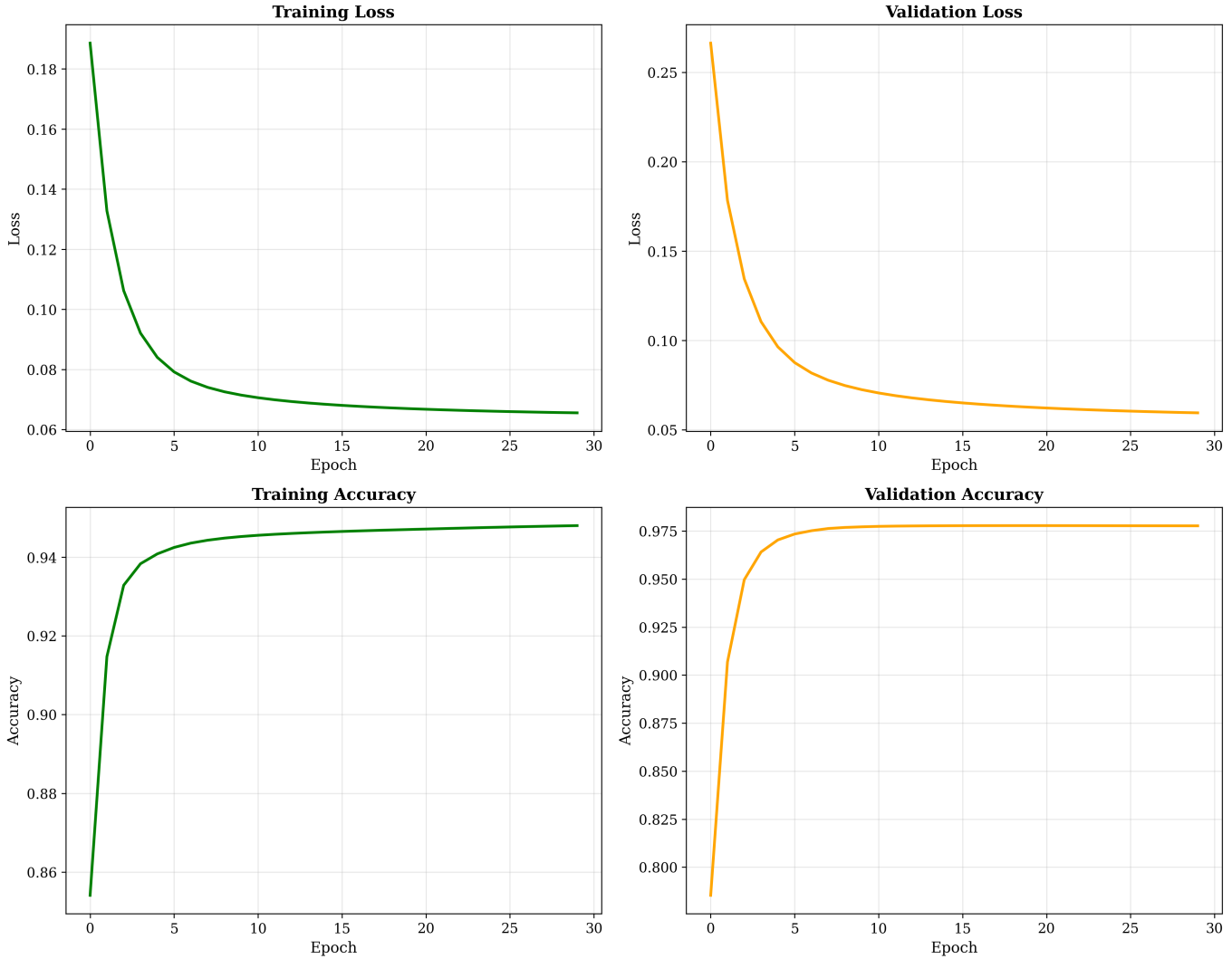
❖ Step 8: Evaluation

```
# Val set
val_set_acc = compute_accuracy(
    X_val, y_val, theta)
print('Evaluation on validation set:')
print(f'Accuracy: {val_set_acc}')
```

```
Evaluation on validation set:
Accuracy: 0.9711386538394017
```

```
# Test set
test_set_acc = compute_accuracy(
    X_test, y_test, theta)
print('Evaluation on test set:')
print(f'Accuracy: {test_set_acc}')
```

```
Evaluation on test set:
Accuracy: 0.9721568765141674
```



Exercise 2

Exercise 2

❖ Sentiment Analysis

➢ Given **Twitter Sentiment Analysis** dataset, build a Softmax Regression model to determine whether a tweet (text) has a positive, neutral or negative sentiment.



Positive



Neutral



Negative

Exercise 2

❖ Framework

➢ In this exercise, we will implement Softmax Regression with two approaches: **NumPy** and **PyTorch.**



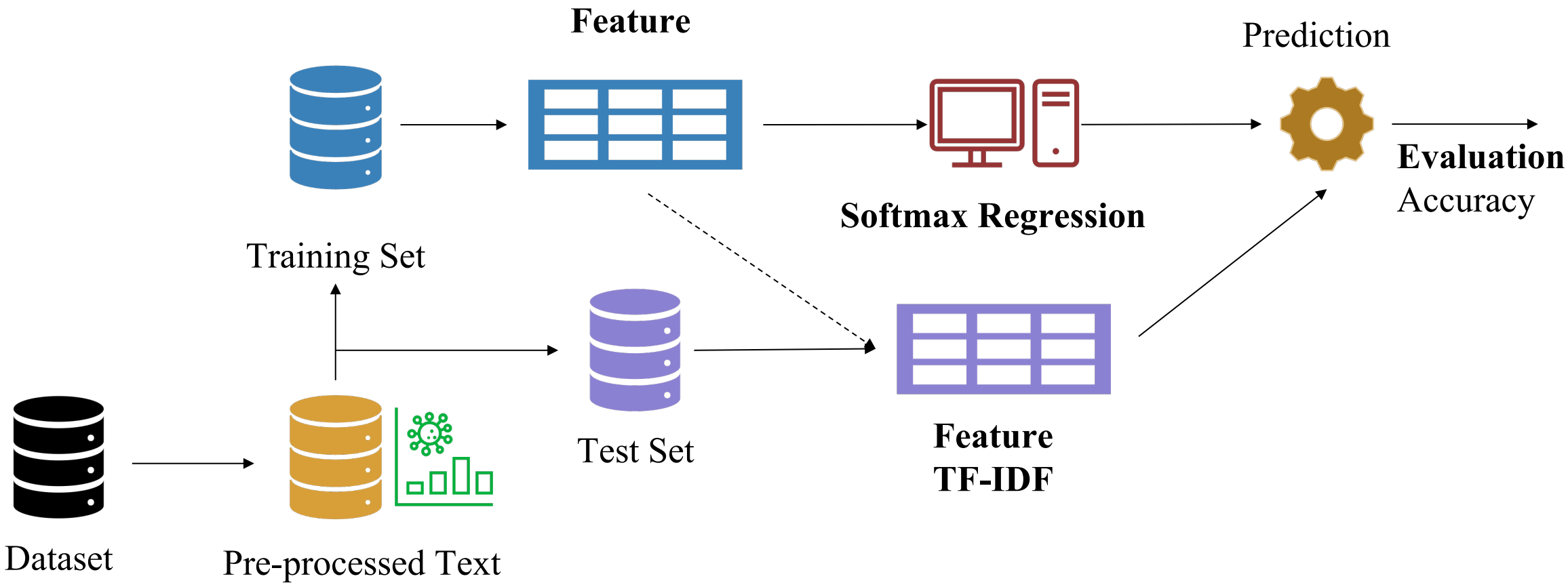
Approach 1: Build with NumPy



Approach 2: Build with PyTorch

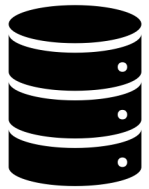
Exercise 2

❖ Pipeline



Exercise 2

❖ Step 1: Load Dataset



Dataset

```
1 dataset_path = '../content/Twitter_Data.csv'
2 df = pd.read_csv(
3     dataset_path
4 )
5 df
```

	clean_text	category
0	when modi promised “minimum government maximum...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0

Exercise 2

❖ Step 2: Get Dataset Information

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 162980 entries, 0 to 162979
Data columns (total 2 columns):
 #   Column      Non-Null Count   Dtype
---  ------      --------------   -----
 0   clean_text  162976 non-null  object
 1   category    162973 non-null  float64
dtypes: float64(1), object(1)
memory usage: 2.5+ MB
```

	category
count	162973.000000
mean	0.225436
std	0.781279
min	-1.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

Exercise 2

❖ Step 3: Text Normalization

	clean_text	category
0	when modi promised “minimum government maximum...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0
...	...	...
162975	why these 456 crores paid neerav modi not reco...	-1.0
162976	dear rss terrorist payal gawar what about modi...	-1.0
162977	did you cover her interaction forum where she ...	0.0
162978	there big project came into india modi dream p...	0.0
162979	have you ever listen about like gurukul where ...	1.0

162980 rows × 2 columns

Raw Text: We love this! Would you go? #talk #makememories #unplug #relax #iphone #smartphone #wifi #connect... http://fb.me/6N3LsUpCu

↓

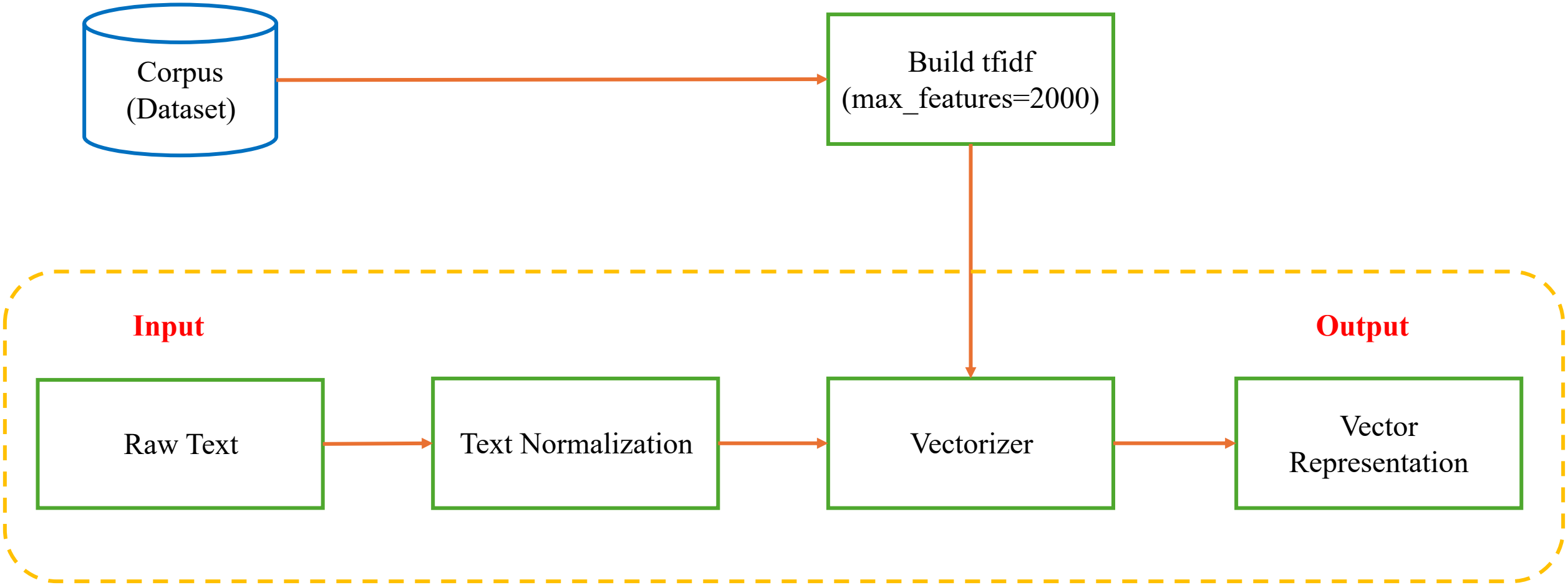
Data Normalization

↓

Vector Representation: *array([0.2, 0.4, 0.11, 0.59, 0.7])*

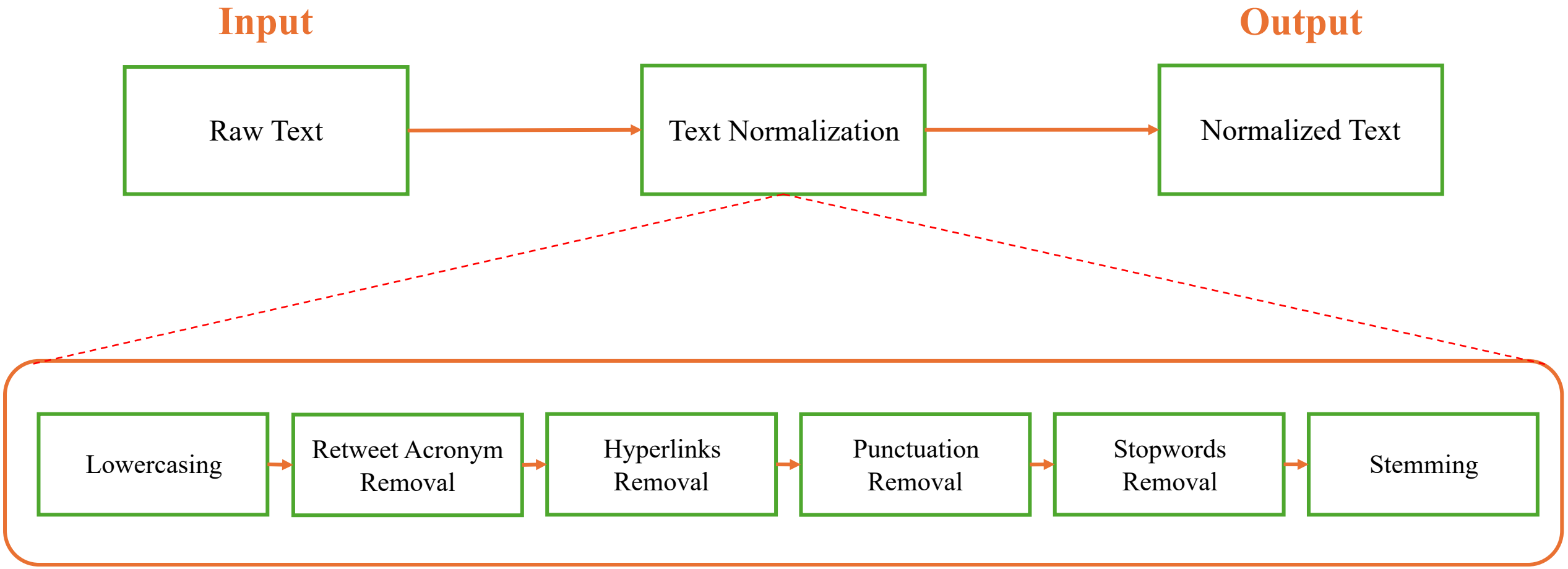
Exercise 2

❖ Step 3: Text Normalization



Exercise 2

❖ Step 3: Text Normalization



Exercise 2

❖ Step 3: Text Normalization

Lowercasing: Convert text into lowercase.

```
1 text = 'Hello WORLD'
2 text = text.lower()
3 print(text)
```

```
hello world
```

Retweet Acronym Removal: Remove "RT" at the start of the string.

```
1 text = "RT Hello World"
2 text = re.sub(r'^RT[\s]+', '', text
3 print(text)
```

```
Hello World
```

Punctuation Removal: Remove all punctuations symbols "#".

```
1 text = "#xinchao, #hello, #konnichiwa"
2 text = re.sub(r'[^\w\s]', '', text)
3 print(text)
```

```
xinchao hello konnichiwa
```

Hyperlinks Removal: Remove all hyperlinks in the string.

```
1 text = "http://fb.me/6N3LsUpCu"
2 text = re.sub(
3     r'https?:\/\/.*[\r\n]*',
4     '',
5     text
6 )
7 print(text)
```

Exercise 2

❖ Step 3: Text Normalization

```
> stopwords("english")
  [1] "i"          "me"         "my"         "myself"     "we"
  [6] "our"        "ours"       "ourselves"  "you"        "your"
 [11] "yours"      "yourself"   "yourselves" "he"         "him"
 [16] "his"        "himself"    "she"        "her"        "hers"
 [21] "herself"    "it"         "its"        "itself"     "they"
 [26] "them"       "their"      "theirs"     "themselves" "what"
 [31] "which"      "who"        "whom"       "this"       "that"
 [36] "these"      "those"      "am"         "is"         "are"
 [41] "was"        "were"       "be"         "been"       "being"
 [46] "have"       "has"        "had"        "having"     "do"
```

```python
text = 'Hello world we are one'
stop_words = set(stopwords.words('english'))
words = text.split()
words = [
    word for word in words \
        if word not in stop_words
]
text = ' '.join(words)
print(text)
```

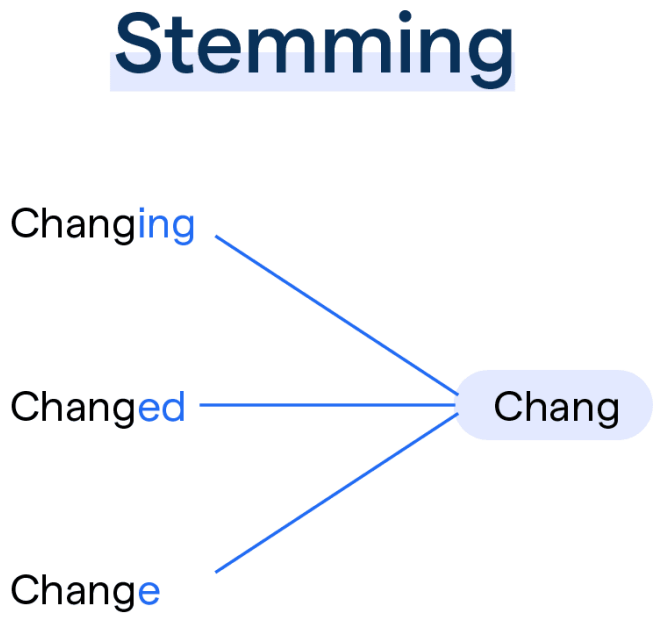
```
Hello world one
```

Stopwords Removal: Remove all “stopwords” which are not necessary to represent in vocabulary.

https://www.researchgate.net/figure/List-of-Stopwords-in-Text-Mining-Library-of-R_fig8_318138987

Exercise 2

❖ Step 3: Text Normalization




```
1 text = 'I am eating breakfast'
2 stemmer = SnowballStemmer('english')
3 words = text.split()
4 words = [stemmer.stem(word) for word in words]
5 text = ' '.join(words)
6 print(text)
```

```
i am eat breakfast
```

Stemming: Convert all the words into its root form (only use in English or similar languages).

https://medium.com/thedeephub/nlp-text-preprocessing-part-3-stemming-lemmatization-a3362789d3a7

Exercise 2

❖ Step 3: Text Normalization

```python
def text_normalize(text):
    # Lowercasing
    text = text.lower()
    # Retweet old acronym "RT" removal
    text = re.sub(r'^rt[\s]+', '', text)
    # Hyperlinks removal
    text = re.sub(r'https?:\/\/.*[\r\n]*', '', text)
    # Punctuation removal
    text = re.sub(r'[^\w\s]', '', text)
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    words = text.split()
    words = [word for word in words if word not in stop_words]
    text = ' '.join(words)
    # Stemming
    stemmer = SnowballStemmer('english')
    words = text.split()
    words = [stemmer.stem(word) for word in words]
    text = ' '.join(words)
    return text
```

```python
text = """We love this! Would you go?
#talk #makememories #unplug
#relax #iphone #smartphone #wifi #connect..
http://fb.me/6N3LsUpCu
"""
text = text_normalize(text)
text
```

```
'love would go talk makememori unplug relax :
```

```python
df['clean_text'] = df['clean_text'].apply(
    lambda x: text_normalize(x)
)
```

Exercise 2

❖ Step 4: Text Encoding – TF-IDF

$$w_{x,y} = tf_{x,y} \times \log \frac{N}{df_x}$$

In which:

- $tf_{x,y}$: frequency of word x in document y.
- df_x: number of documents containing word x.
- N: total number of documents.

```
1 vectorizer = TfidfVectorizer(
2     max_features=2000
3 )
4 X = vectorizer.fit_transform(
5     df['clean_text']
6 ).toarray()
```

```
1 print(X.shape)
```

```
(162969, 2000)
```

AI VIET NAM
@aivietnam.edu.vn

Exercise 2

```
term_document_matrix = {}
for (title, article) in tqdm(doc_lists):
  vec = vectorize(article, vocab)
  term_document_matrix[(title, article)] = vec
```

❖ Step 4: Text Encoding – TF-IDF

doc1 = "Học sách học AI."

doc2 = "Sách Học Máy"

doc3 = "Người ấy là ai?"

Normalize & Tokenize

doc1 = ['học', 'sách', 'học', 'ai']

doc2 = ['sách', 'học', 'máy']

doc3 = ['người', 'ấy', 'là', 'ai']

Create Vocabulary

Vocab = ['học', 'sách', 'ai', 'máy', ' người', 'ấy', là']

	học	sách	ai	máy	người	ấy	là
doc1	0	0	0	0	0	0	0
doc2	0	0	0	0	0	0	0
doc3	0	0	0	0	0	0	0

Each row is also **document index**

Exercise 2

❖ Step 4: Text Encoding – TF-IDF

	học	sách	ai	máy	người	ấy	là
doc1	0	0	0	0	0	0	0
doc2	0	0	0	0	0	0	0
doc3	0	0	0	0	0	0	0

Document-term matrix

Tf-idf score formula for a word x in document y:

$$w_{x,y} = tf_{x,y} \times \log \frac{N}{df_x}$$

In which:

- $tf_{x,y}$: frequency of word x in document y.
- df_x: number of documents containing word x.
- N: total number of documents.

Exercise 2

❖ Step 5: One-hot Encoding Label

Label encoding

Label
0
0
1
3
2
1
2



One-hot encoding

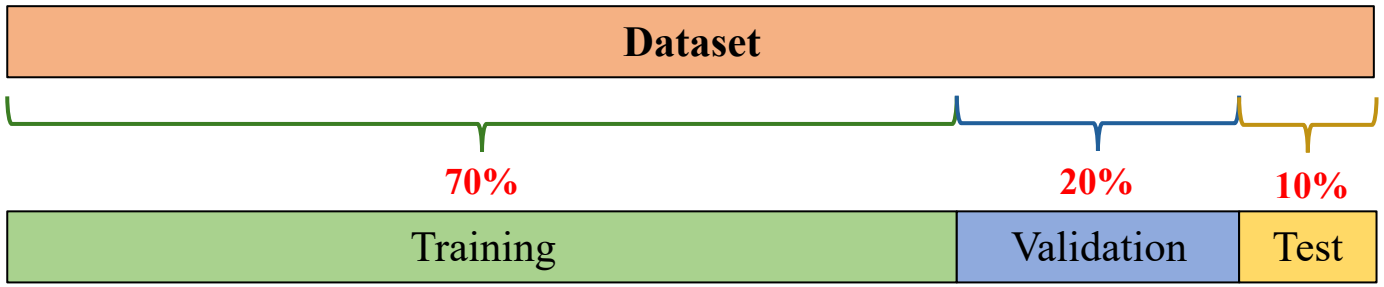
Label			
1	0	0	0
1	0	0	0
0	1	0	0
0	0	0	1
0	0	1	0
0	1	0	0
0	0	1	0

```
n_classes = df['category'].nunique()
n_samples = df['category'].size

y = df['category'].to_numpy() + 1
y = y.astype(np.uint8)
y_encoded = np.array(
    [np.zeros(n_classes) for _ in range(n_samples)]
)
y_encoded[np.arange(n_samples), y] = 1
```

Exercise 2

❖ Step 6: Split Train, Val, Test Set




```
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y_encoded, dtype=torch.float32)
```

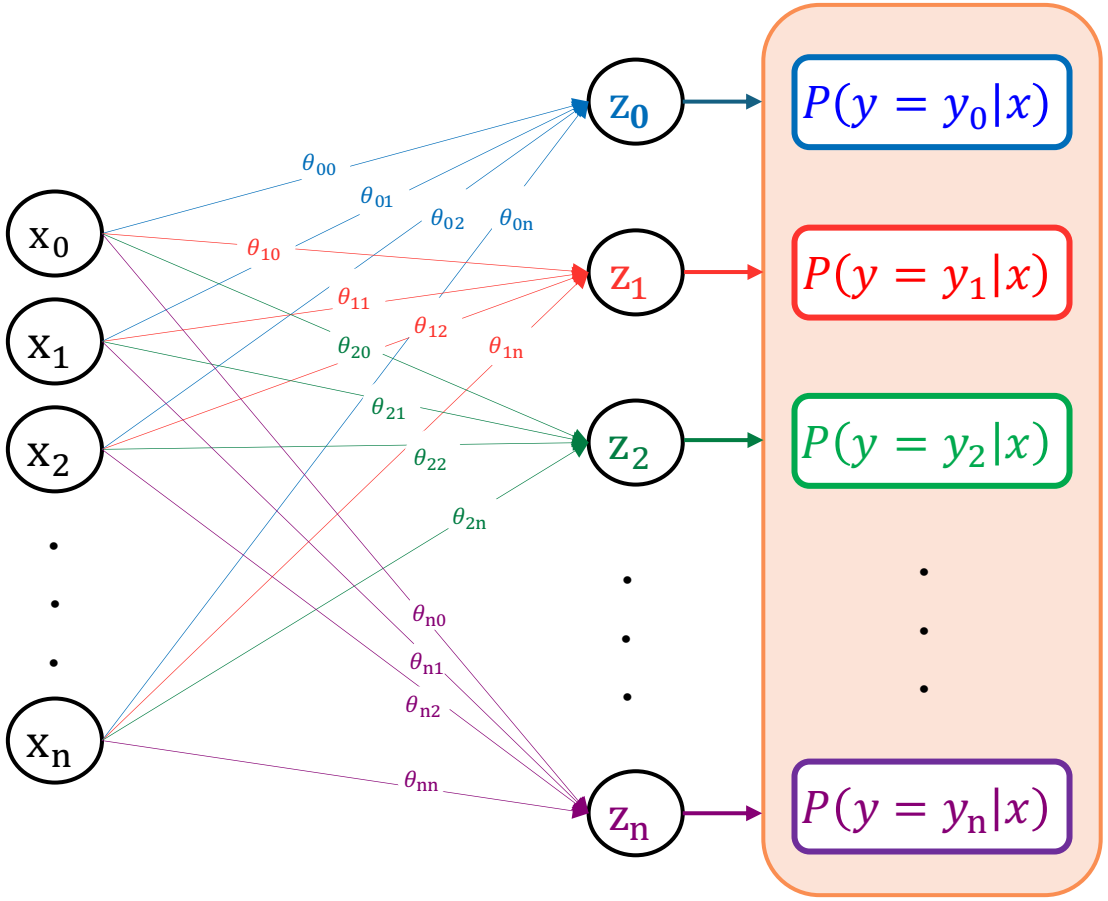
```
val_size = 0.2
test_size = 0.125
random_state = 2
is_shuffle = True
```

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=val_size,
    random_state=random_state,
    shuffle=is_shuffle
)

X_train, X_test, y_train, y_test = train_test_split(
    X_train, y_train,
    test_size=test_size,
    random_state=random_state,
    shuffle=is_shuffle
)
```

Exercise 2

❖ Step 7: Softmax Regression Model




```python
class SoftmaxRegression(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(SoftmaxRegression, self).__init__()
        self.linear = nn.Linear(
            input_dim,
            output_dim,
            bias=True
        )

    def forward(self, x):
        return self.linear(x)
```

Exercise 2

❖ Step 8: Training

```python
lr = 0.1
epochs = 500
torch.manual_seed(random_state)
if torch.cuda.is_available():
    torch.cuda.manual_seed(random_state)

input_dim = X_train.shape[1]
output_dim = y_train.shape[1]

model = SoftmaxRegression(
    input_dim, output_dim
)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(
    model.parameters(), lr=lr
)
```

```python
for epoch in range(epochs):
    model.train()

    # Zero the gradients
    optimizer.zero_grad()

    # Forward pass
    y_hat = model(X_train)

    # Compute loss
    train_loss = criterion(y_hat, y_train)

    train_losses.append(train_loss.item())

    train_acc = compute_accuracy(y_hat, y_train)
    train_accs.append(train_acc)

    # Backward pass and optimization
    train_loss.backward()
    optimizer.step()
```

Exercise 2

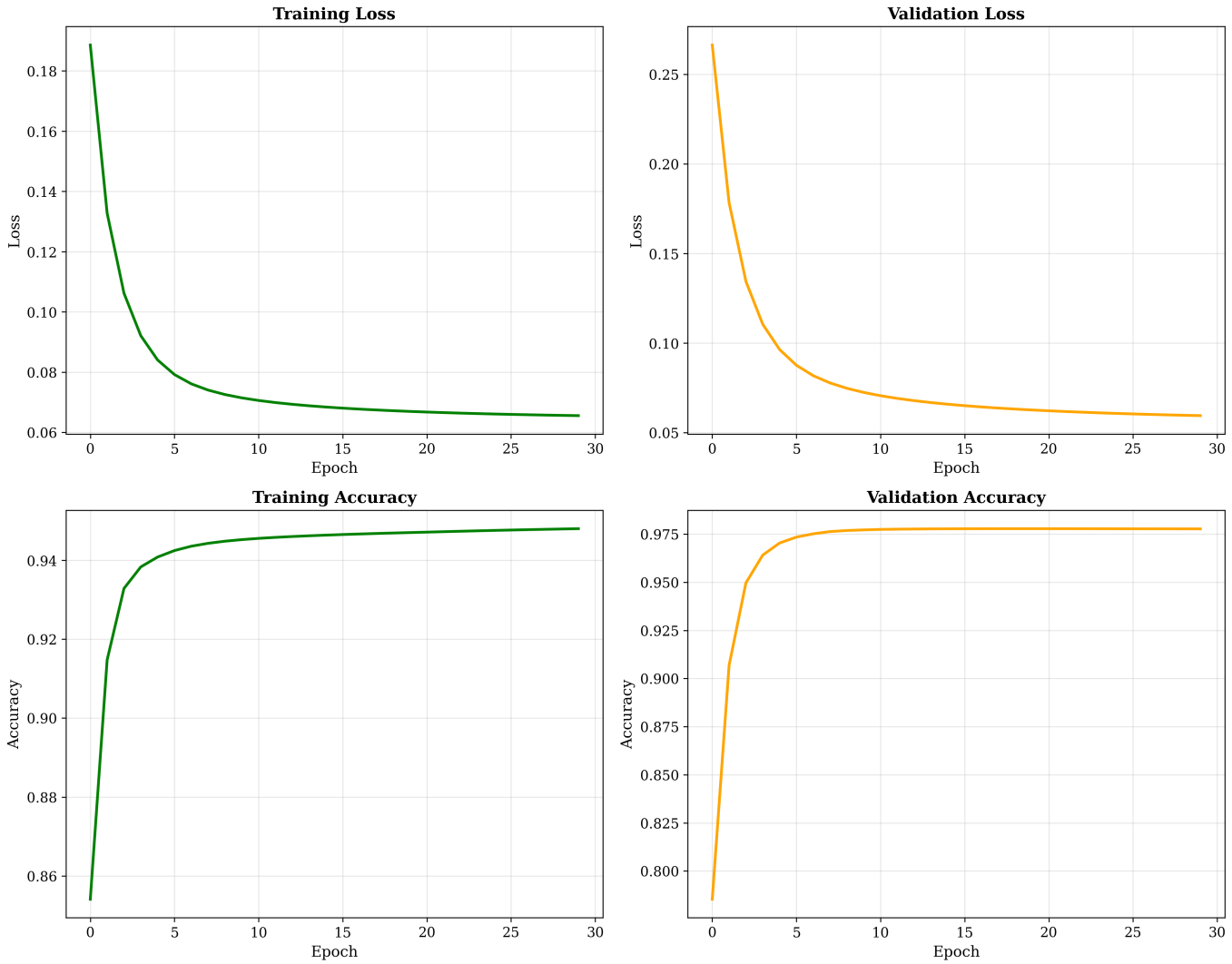
❖ Step 9: Evaluation

```python
# Val set
model.eval()
with torch.no_grad():
    y_hat = model(X_val)
    val_set_acc = compute_accuracy(y_hat, y_val)
    print('Evaluation on validation set:')
    print(f'Accuracy: {val_set_acc}')
```

```
Evaluation on validation set:
Accuracy: 0.4506350862121863
```

```python
# Test set
model.eval()
with torch.no_grad():
    y_hat = model(X_test)
    test_set_acc = compute_accuracy(y_hat, y_test)
    print('Evaluation on test set:')
    print(f'Accuracy: {test_set_acc}')
```

```
Evaluation on test set:
Accuracy: 0.4491010615450696
```

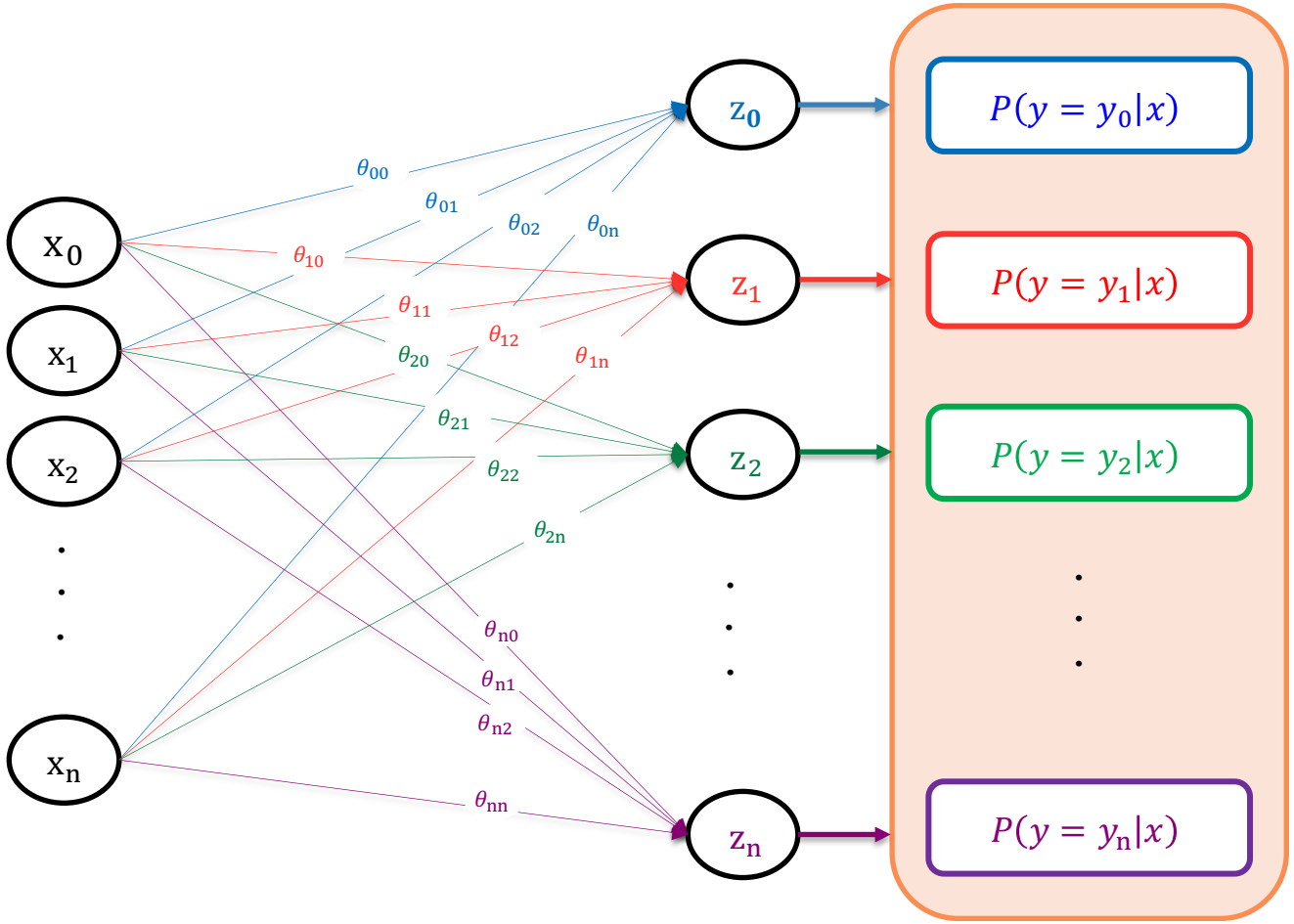


AI VIET NAM
@aivietnam.edu.vn

KAHOOT QUIZ

Summarization and Q&A

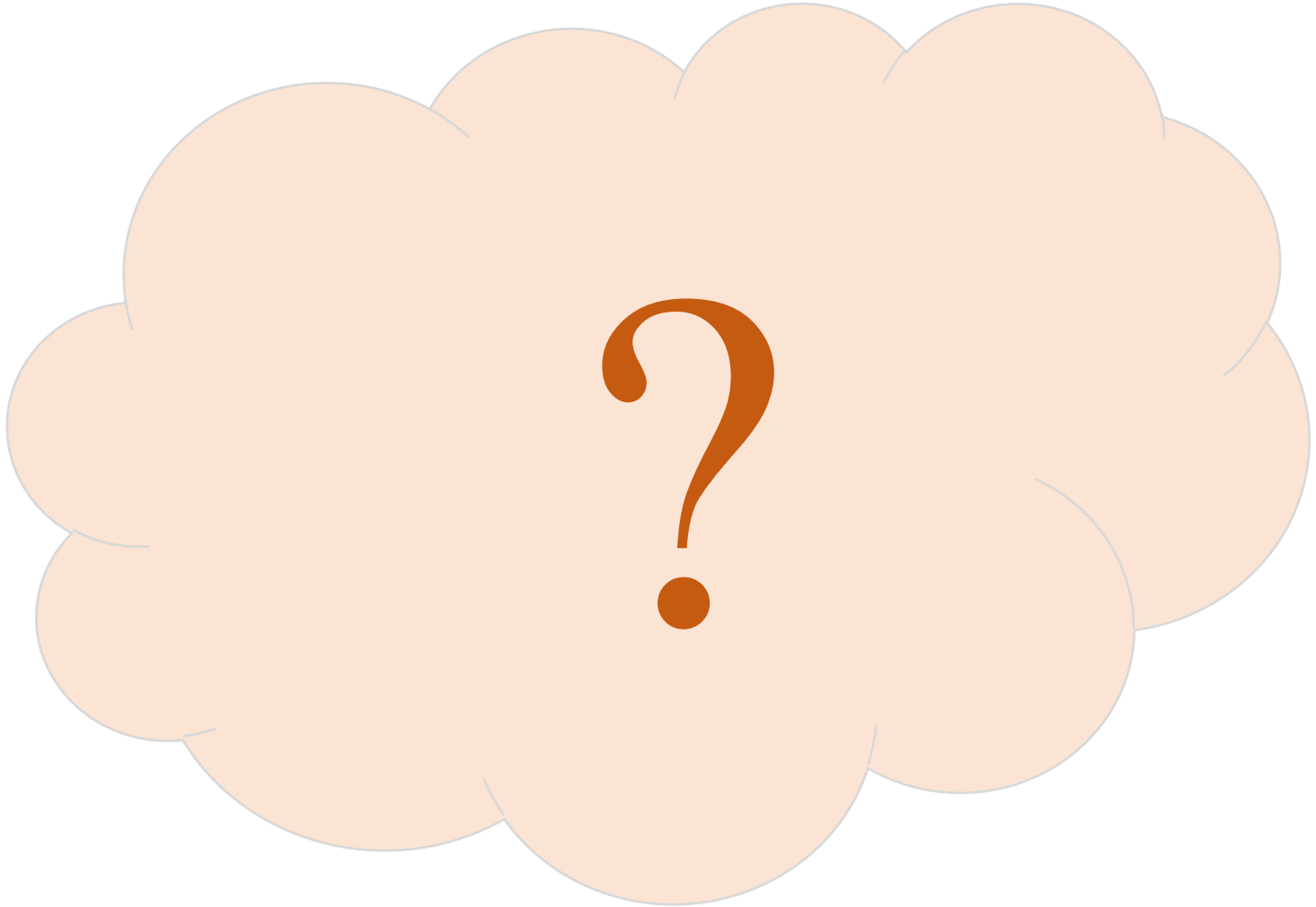
Summarization and Q&A



In this session, we have discussed:

- How Softmax Regression extends Logistic Regression to multiclass classification.
- How the Softmax Function and SGD are used to train Softmax Regression.
- How Softmax Regression is implemented for Card Fraud Detection and Sentiment Analysis using NumPy and PyTorch.

Summarization and Q&A




Thank you!